A

Major Project Report

on

ENHANCING BUILDING SAFETY THROUGH MACHINE LEARNING -BASED SMOKE DETECTION

Submitted in Partial Fulfillment to Osmania University for the Award of the Degree of

Bachelor of Engineering

In

Department of Computer Science Engineering - AIML

By

M.A. OMER 160922748048

Under the Esteemed Supervision of

Dr. Mohammed Tajuddin

Associate Professor

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING - AIML

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

Approved by AICTE | Affiliated to Osmania University | Estd.2003

Sy.No.32, Himayat Sagar, Near TGPA Junction, Hyderabad-500091, India.

(2025-2026)

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

Approved by AICTE | Affiliated to Osmania University | Estd. 2003

Accredited with 'A' grade by NAAC | Accredited by NBA

Department of Computer Science Engineering - AIML

CERTIFICATE

This is to certify that Major Project entitled **"ENHANCING BUILDING SAFETY THROUGH MACHINE LEARNING-BASED SMOKE DETECTION"** is a Bonafide record of the work successfully completed and submitted by

M.A. OMER 160922748048

Under the Supervision of **Dr. Mohammed Tajuddin**, **Associate Professor**, Department of Computer Science Engineering - AIML for the requirement of partial fulfillment for the award of degree of Bachelor of Engineering in Computer Science Engineering - AIML during the academic year 2025-2026 from the Lords Institute of Engineering and Technology, Hyderabad.

Project Guide
Dr. Mohammed Tajuddin
Associate Professor

Head of the Department
Dr. Abdul Rasool MD
HOD

Principal
Dr. Ravi Kishore Singh

External Examiner
Date:

DECLARATION

We hereby declare that Major Project entitled **"ENHANCING BUILDING SAFETY THROUGH MACHINE LEARNING-BASED SMOKE DETECTION"** is being submitted by us in partial fulfilment to Osmania University for the award of Bachelor of Engineering in the Department of Computer Science Engineering - AIML at the Lords Institute of Engineering and Technology (Autonomous),Telangana-500091,is the result of investigations carried out by us under the Guidance of **Dr. Mohammed Tajuddin**, **Associate Professor**, Department of Computer Science Engineering - AIML, Lords Institute of Engineering and Technology.

The work is original and has not been submitted for any Degree or Diploma for this or any other university.

M.A. OMER 160922748048

ACKNOWLEDGEMENT

M.A. OMER

(160922748048)

+91 9652159548

mohammedabdulomer99@gmail.com

In the name of almighty, the most beneficent and the most merciful, we thank the lord for helping us in all the stages of this thesis work.

We would like to thank our guide **Dr. Mohammed Tajuddin**, **Associate Professor**, for their aspiring guidance and friendly advice during the project work. He was a constant motivation throughout the project and took valuable time whenever it was required, for which we are very grateful to him.

We would like to thank **Dr. Abdul Rasool MD**, **Head of the Department** of Computer Science Engineering - AIML, for his support throughout our project. It gives us great pleasure in acknowledging the support and help from him throughout the project.

Our experience here has been full of opportunities to learn from faculty with a wide and deep expertise in Engineering. We would like to thank our beloved **Principal Dr. Ravi Kishore Singh**. And those in the department who were particularly instrumental in contributing to our experience at Lords Institute of Engineering and Technology (A).

We thank both teaching and non-teaching staff members of the Computer Science Engineering - AIML department for their cooperation and support throughout the duration of this project.

Lastly, we drive great pleasure and we would like to especially thank our parents for their patience, continuous encouragement, support and love given towards completion of our research work.

Vision of the Institute:

Lords Institute of Engineering and Technology strives for excellence in professional education through quality, innovation and teamwork and aims to emerge as a premier institute in the state and across the nation.

Mission of the Institute:

- To impart quality professional education that meets the needs of present and emerging technological world.
- To strive for student achievement and success, preparing them for life, career and leadership.
- To provide a scholarly and vibrant learning environment that enables faculty, staff and students to achieve personal and professional growth.
- To contribute to advancement of knowledge, in both fundamental and applied areas of engineering and technology.
- To forge mutually beneficial relationships with government organizations, industries, society and the alumni.

Principal

PRINCIPAL
Lords Institute of Engineering & Tech.
(An Autonomous Institution)
Sy. No. 32, Himayath Sagar, Hyderabad-500091.T.S.

Vision of the Department:

To obtain high quality standards in education by utilizing cutting-edge technologies, build an ecosystem that will contribute meaningfully to society by producing world class Engineers in the field of Computer Science Engineering - AIML

Mission of the Department:

DM1: To Provide intense training to generate knowledge using state of art Artificial Intelligence and Machine Learning concepts and technologies.

DM2: To bring along scholars and students in a interdisciplinary setting to carry on making substantial advances in expanding the potential of a machine or human-machine systems that can handle challenging higher cognitive functions.

DM3: To encourage the formation of academic-industry collaborations and societal outreach projects.

DM4: To educate students with modern tools to take part fully and ethically in a varied society while also encouraging lifelong learning.

Note: DM: Department Mission

Head of the Department

Head of the Department
CSM (CSE-AI&ML)
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-500091.

B.E. COMPUTER SCIENCE ENGINEERING - AIML Program Educational Objectives (PEOs):

PEO1	To establish themselves as successful professionals with strong fundamental knowledge in basic and engineering sciences to find suitable solutions of technological and real-life challenges using innovative tools.
PEO2	To enhances technical competency and problems solving skills through state of art facilities for adequate solutions to technical problems.
PEO3	Acquire high skill-set by continuous training, multi-disciplinary activities, team work, effective communications, Information Technology tools usage and ethics so that students shall acquire good job opportunities and also will help in their higher education.
PEO4	Giving consultancy services to industrial, societal challenges and promoting department-industry interactions, by enhancing technical, managerial, environmental responsibilities and lifelong learning with sustainable development.

Head of the Department

Head of the Department
CSM (CSE-AI&ML)
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-500091.

B.E. COMPUTER SCIENCE ENGINEERING - AIML Program Outcomes (POs):

Engineering Graduates will be able to:

1.	**PO1: Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
2.	**PO2: Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.
3.	**PO3: Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
4.	**PO4: Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
5.	**PO5: Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
6.	**PO6: The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety.

7.	**PO7: Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
8.	**PO8: Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams
9.	**PO9: Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.
10.	**PO10: Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, and to manage projects and in multidisciplinary environments.
11.	**PO11: Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Head of the Department

Head of the Department
CSM (CSE-AI&ML)
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-500091.

B.E. COMPUTER SCIENCE ENGINEERING – AIML Program Specific Outcomes (PSO's):

PSO1	**Professional Skills:** An ability to understand the basic concepts of AI & ML to apply them to various areas, like production, Business Automation, designing etc., in the design and implementation of complex systems.
PSO2	**Problem-Solving Skills:** An ability to solve complex Computer Science Engineering problems, using latest AI & ML tools, along with analytical skills to arrive cost effective and appropriate solutions.

Head of the Department

Head of the Department
CSM (CSE-AI&ML)
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-500091.

Course Outcomes: C85 – Major Project

Student will be able to:

CO. No	Description	Blooms Taxonomy Level
C85.1	Demonstrate the ability to synthesize and apply the knowledge and skills acquired in the academic program to real-world problems.	BTL 5 (Synthesis)
C85.2	Evaluate different solutions based on economic and technical feasibility	BTL 6 (Evaluation)
C85.3	Effectively plan a project and confidently perform all aspects of project management	BTL 4 (Analysis)
C85.4	Demonstrate effective oral communication skills	BTL 3 (Application)
C85.5	Demonstrate effective team work and written skills	BTL 3 (Application

Course Articulation Matrix:

Mapping of Course Outcomes (CO) with Program Outcomes (PO) andProgram Specific Outcomes (PSO's):

Course Outcome s (CO)	Program Outcomes (PO)											Program Specific Outcomes (PSO's)	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
C85.1	3	3	2	3	3	2	--	3	3	3	3	3	3
C85.2	3	3	3	3	3	2	--	3	3	3	3	2	2
C85.3	2	3	1	--	2	1	--	3	2	3	3	2	2
C85.4	3	3	--	3	3	1	--	2	2	3	3	1	2
C85.5	2	1	2	--	2	2	--	1	--	3	3	3	1
Average	**2.6**	**3**	**2**	**3**	**2.6**	**1.6**	**--**	**2.4**	**2.5**	**3**	**3**	**2.2**	**2**

Level:

1- Low correlation (Low), 2- Medium correlation (Medium), 3-High correlation(High)

SDG Mapping:

SDG	Mapped Indicator	SDG	Mapped Indicator	SDG	Mapped Indicator
1 NO POVERTY		7 AFFORDABLE AND CLEAN ENERGY		13 CLIMATE ACTION	✓
2 ZERO HUNGER		8 DECENT WORK AND ECONOMIC GROWTH		14 LIFE BELOW WATER	
3 GOOD HEALTH AND WELL-BEING	✓	9 INDUSTRY, INNOVATION AND INFRASTRUCTURE	✓	15 LIFE ON LAND	
4 QUALITY EDUCATION		10 REDUCED INEQUALITIES		16 PEACE, JUSTICE AND STRONG INSTITUTIONS	
5 GENDER EQUALITY		11 SUSTAINABLE CITIES AND COMMUNITIES	✓	17 PARTNERSHIPS FOR THE GOALS	
6 CLEAN WATER AND SANITATION		12 RESPONSIBLE CONSUMPTION AND PRODUCTION			

ABSTRACT

Fire and smoke pose a severe threat to human life and property in building environments. Traditional smoke detection systems rely on single-sensor threshold triggers that are prone to false alarms and missed early-stage detections. This project, Enhancing Building Safety through Machine Learning and Deep Learning-Based Smoke Detection, presents a dual-module intelligent detection system that combines classical machine learning with modern deep learning to provide accurate, reliable, and comprehensive smoke detection. The first module applies seven machine learning classifiers — Random Forest, Gradient Boosting, AdaBoost, Logistic Regression, Support Vector Machine, Decision Tree, and K-Nearest Neighbours — to a real-world IoT sensor dataset containing 62,630 readings across 13 environmental sensor channels. Five classifiers (Random Forest, Gradient Boosting, Decision Tree, KNN, and SVM) achieve perfect or near-perfect scores across all metrics. AdaBoost reaches 99.97% and Logistic Regression 99.31% precision on the test set. The second module uses transfer learning with MobileNetV2, a lightweight convolutional neural network pre-trained on ImageNet, to visually detect fire and smoke in uploaded images. The CNN was trained on the Kaggle Fire Dataset (755 fire images and 244 non-fire images) and achieves 96.98 percent validation accuracy. GPU-accelerated training on an NVIDIA RTX 3050 Ti reduced training time by a factor of 2.3 compared to CPU. Additionally, a YOLOv8 object detection model (best.pt) has been integrated into the same CNN prediction page to draw bounding boxes around detected fire and smoke regions, providing localised visual detection alongside the CNN classification result. Both modules are integrated into a unified full-stack web application built using the Django framework, providing separate role-based portals, an interactive IoT dataset browser, on-demand model training, sensor-based prediction, image-based CNN fire classification, and YOLOv8 bounding-box fire and smoke detection on the same prediction page.

Keywords: Machine Learning, Deep Learning, Smoke Detection, Fire Detection, IoT Sensor Data, Convolutional Neural Network (CNN), MobileNetV2, YOLOv8, Object Detection, Computer Vision, Smart Building Safety.

LIST OF FIGURES

Fig. No	Title	Page No.
1.1	System Architecture Overview – Dual-Module Smoke Detection System	6
1.2	Proposed Fire and Smoke Detection System Using AI and IoT Integration	8
1.3	Wildfire smoke-detection operation flowchart using deep learning.	10
3.1	Dataset Preprocessing Pipeline Flowchart	21
3.2	Fire Alarm Class Distribution	24
3.3	Data Preprocessing Workflow for Smoke Detection System	25
4.1	ML Classifier AUC-ROC Performance Comparison	27
4.2	Confusion Matrix of Random Forest Classifier	31
5.1	MobileNetV2 Transfer Learning Architecture for Fire/Smoke Detection	33
5.2	Two-Phase CNN Architecture Flow for Fire using MobileNetV2	34
6.1	Django MVT Application Architecture and Data Flow Diagram	38
6.2	YOLOv8 Fire/Smoke Detection Output	40
7.1	User Registration and Login Workflow Diagram	47
8.1	Home page	60
8.2	User Registration Interface	61
8.3	Admin Login Page	61
8.4	Admin Dashboard	62
8.5	User Activation Page	62
8.6	User Login Page	62
8.7	User Home Page	63
8.8	Dataset View	63
8.9	Model Training Interface with Performance Charts	64
8.10	Sensor Prediction Interface - No Smoke Detected	64
8.11	Sensor Prediction Interface - Smoke Detected	65
8.12	CNN Prediction Interface - Smoke/Fire Detected	65
8.13	CNN Prediction Interface - No Smoke/Fire Detected	66
8.14	YOLO Prediction Page Output with Confidence Scores	66
8.15	CNN Training and Validation Accuracy Curve over 10 Epochs	67

LIST OF TABLES

Table No.	Title	Page No.
3.1	IoT Sensor Dataset Summary	22
3.2	Input Feature Columns	23
3.3	CNN Image Dataset Summary	26
4.1	ML Model Performance Results	31
4.2	5-Fold Cross-Validation AUC-ROC Results	32
5.1	CNN Training Configuration	35
5.2	CNN Training Results	36
6.1	Technology Stack	41
6.2	Application URL Routes	42
7.1	Key Project Files	48
8.1	Final ML Model Performance Summary	67
8.2	CNN Module Performance Summary	68
8.3	Development and Testing Environment	69
8.4	Sample Test Cases	71

TABLE OF CONTENTS

CONTENT		PAGE NO
Vision and Mission of the Institute		i
Vision and Mission of the Department		ii
Program Educational Objectives		iii
Program Outcomes		iv
Program Specific Outcomes		vi
Course Outcomes		vii
Course Articulation Matrix		viii
Abstract		ix
List of Figures		x
List of Tables		xi
CHAPTER 1: INTRODUCTION		**1 – 10**
1.1	Overview	1
1.2	Problem Statement	1
1.3	Objectives	2
1.4	Scope of the Research	2
1.5	Significance of the Report	3
1.6	Justification of the Study	4
1.7	Limitations of the Research	5
1.8	System Architecture and Functional Modules	5
1.9	Evolution of Building Safety in Smoke Detection	7
1.10	Applications of the Proposed System	8
1.11	General Working Principle of the Proposed System	9
1.12	Organisation of the Report	10
Chapter 2: LITERATURE REVIEW		**11 – 20**
2.1	Artificial Intelligence-Based Real-Time Smoke and Fire Detection and Security Management Algorithms (2025)	11
2.2	Real-Time Fire Object Detection System Using Machine Learning (2025)	13

2.3 A Review on Fire and Smoke Detection with Intelligent Control for Enhanced Safety Using Machine Learning (ML) and Internet of Things (IoT) (2024) 15

2.4 2.4 Enhancing Fire Detection in Smart Buildings Using Deep Learning and IoT (2023) 17

2.5 2.5 Enhancing Building Safety Through Machine Learning-Based Smoke Detection (2022) 19

Chapter 3: DATASET AND DATA PREPROCESSING **21 – 26**

3.1 ML Module Dataset 21

3.1.1 Dataset Summary 22

3.1.2 Feature Columns 22

3.1.3 Class Distribution Analysis 24

3.2 Data Preprocessing 25

3.3 CNN Module Dataset 25

CHAPTER 4: ML SYSTEM WORKFLOW **27 – 32**

4.1 Introduction to the ML Module 27

4.2 Algorithms Used 27

4.2.1 Random Forest 27

4.2.2 Gradient Boosting 28

4.2.3 AdaBoost 28

4.2.4 Logistic Regression 28

4.2.5 Support Vector Machine 28

4.2.6 Decision Tree 28

4.2.7 K-Nearest Neighbours 28

4.3 Training Process 29

4.4 Evaluation Metrics 29

4.4.1 Precision 29

4.4.2 Recall 29

4.4.3 AUC-ROC Score 30

4.4.4 Intersection Over Union (IoU) 30

4.5 Model Performance Results 30

4.6 5-Fold Cross-Validation 32

4.7 Prediction Workflow 32

CHAPTER 5: DEEP LEARNING MODULE **33 – 37**

5.1 Introduction to the DL Module 33
5.2 Convolutional Neural Networks 33
5.3 Transfer Learning with MobileNetV2 34
5.4 Model Architecture 34
5.5 Training Process 35
5.6 Training Results 36
5.7 Prediction Workflow 37
CHAPTER 6: SYSTEM DESIGN AND ARCHITECTURE 38 – 46
6.1 System Overview 38
6.2 Application Modules 39
6.2.1 User Registration and Login 39
6.2.2 Admin Dashboard 39
6.2.3 Dataset View 39
6.2.4 ML Training Module 39
6.2.5 ML Prediction Module 39
6.2.6 CNN + Yolo Prediction Module 39
6.3 Technology Stack 41
6.4 URL Route Architecture 42
6.5 File and Directory Structure 42
6.6 System Analysis 43
6.6.1 Existing System 43
6.6.2 Proposed System 44
6.7 Input and Output Design 45
6.7.1 Input Design 45
6.7.2 Output Design 45
6.8 Software Environment 45
6.8.1 Python 45
6.8.2 Django Framework 46
6.8.3 Hardware Requirements 46
CHAPTER 7: IMPLEMENTATION 47 – 59
7.1 Project File Structure 47
7.2 Database Model 49
7.3 User Registration Form 49
7.4 URL Configuration 50

7.5 Admin Views 51
7.6 User Views: Training Module 52
7.7 User Views: Prediction Module 53
7.8 CNN Training Script 54
7.9 CNN + YOLO Prediction View: users/views.py 56
7.10 Execution and Workflow 57
7.10.1 Step 1: Environment Setup 57
7.10.2 Step 2: Apply Migrations and Run Server 58
7.10.3 Step 3: Train the CNN Model 58
7.10.4 Step 4: Application Workflow 58
CHAPTER 8: RESULT ANALYSIS **60 – 75**
8.1 System Screenshots 60
8.2 ML Module Results 67
8.3 CNN Module Results 68
8.4 System Capabilities 68
8.5 Hardware and Software Environment 69
8.6 System Testing 70
8.6.1 Unit Testing 70
8.6.2 Integration Testing 70
8.6.3 Functional Testing 70
8.6.4 Sample Test case 71
8.7 Original Contribution 72
8.8 Conclusion 73
8.9 Future Scope 74
REFERENCES **76 – 78**

CHAPTER 1

INTRODUCTION

1.1 Overview

Fire and smoke are among the most devastating causes of loss of life and property in buildings across the world. Early and accurate detection of smoke is critical to preventing casualties and minimising structural damage. Traditional smoke detectors rely on simple threshold-based hardware sensors, which can be prone to false alarms and limited sensitivity in complex real-world environments.

This project, titled Enhancing Building Safety through Machine Learning and Deep Learning Based Smoke Detection, addresses this problem by developing an intelligent, software-based smoke detection system using two complementary approaches. The first applies classical machine learning algorithms to IoT sensor data. The second uses a deep learning convolutional neural network to visually detect fire and smoke in uploaded images.

The entire system is delivered as a full-stack web application built using the Django framework, accessible through any standard web browser without requiring any installation on the user's device.

1.2 Problem Statement

Conventional fire detection systems operate on single-sensor thresholds. A smoke detector may only trigger when particulate matter concentration exceeds a fixed threshold, without considering other environmental signals such as temperature rise, carbon dioxide levels, or volatile organic compound concentrations. This leads to two key problems: false alarms triggered by non-dangerous conditions such as cooking steam, and missed detections in early-stage fires where no single sensor has yet crossed its threshold.

Machine learning models solve this by analysing multiple sensor readings simultaneously and learning complex patterns that distinguish real fire conditions from normal environmental variation. Deep learning models extend this further by enabling visual detection from uploaded photographs or camera images.

1.3 Objectives

The objectives of this project are as follows:

- **Objective 1**: To collect and process a real-world IoT smoke detection sensor dataset containing over 62,000 readings across 13 environmental sensor channels.
- **Objective 2**: To train, evaluate, and compare seven classical machine learning classifiers on the sensor dataset for smoke detection.
- **Objective 3**: To develop and train a MobileNetV2 CNN using transfer learning to detect fire and smoke visually from images.
- **Objective 4**: To integrate both ML and DL modules into a unified, role-based Django web application.
- **Objective 5**: To achieve high detection accuracy and present results through visual charts, metrics tables, and confidence scores.
- **Objective 6**: To integrate YOLOv8 object detection on the same CNN prediction page to draw bounding boxes around detected fire and smoke regions with class labels and confidence scores.

1.4 Scope of the Research

Aim: The primary aim of this project is to design and develop an intelligent smoke detection system that enhances building safety by integrating Machine Learning and Deep Learning techniques for accurate, real-time fire and smoke detection using sensor data and image-based analysis.

Objectives: The objectives of the proposed system include:

- To develop a machine learning module that analyzes multi-sensor IoT data for detecting smoke and fire conditions.
- To implement and compare multiple ML classifiers such as Random Forest, SVM, Gradient Boosting, AdaBoost, Logistic Regression, Decision Tree, and KNN

- To design a deep learning model using MobileNetV2 with transfer learning for image-based fire and smoke detection
- To integrate a YOLOv8 object detection model for real-time localisation of fire and smoke regions using bounding boxes
- To build a full-stack web application using Django for seamless interaction between ML/DL models and users
- To enable prediction using both manual sensor input and uploaded images through a browser-based interface
- To implement real-time model training, evaluation, and visualization using charts and performance metrics
- To ensure the system operates without requiring physical IoT hardware by allowing simulation through dataset inputs.
- To provide a user-friendly interface with role-based access for administrators and users.

1.5 Significance of the Report

The significance of this research lies in its contribution to enhancing building safety through the application of advanced artificial intelligence techniques. Fire and smoke incidents remain one of the leading causes of damage to life and property, especially in densely populated urban environments. Traditional smoke detection systems are limited by their reliance on fixed thresholds and single-sensor inputs, which often result in delayed detection, false alarms, and reduced reliability.

This research introduces an intelligent, data-driven approach by combining Machine Learning and Deep Learning models to improve the accuracy and timeliness of smoke detection. The integration of multi-sensor IoT data analysis allows the system to identify complex environmental patterns that are not detectable using conventional methods. Additionally, the incorporation of image-based detection using a CNN and YOLOv8 enhances the system's ability to visually verify fire and smoke conditions, thereby reducing uncertainty and false positives.

Another significant contribution of this research is the development of a unified web-based platform that integrates both sensor-based and image-based detection

modules. This enables real-time monitoring, easy accessibility, and practical usability in real-world scenarios without requiring specialised hardware during operation. The system also demonstrates scalability and adaptability, allowing it to be extended for smart building applications and future integration with live IoT devices and surveillance systems.

Overall, this research contributes to the advancement of intelligent safety systems by providing a reliable, efficient, and scalable solution for early fire detection. It supports the development of safer environments, aligns with smart city initiatives, and demonstrates the practical application of AI technologies in critical real-world domains.

1.6 Justification of the Study

The need for this study arises from the limitations of conventional smoke detection systems, which rely on fixed threshold values and single-sensor inputs. Such systems often fail to detect early-stage fires and are prone to false alarms caused by non-threatening conditions like dust or cooking fumes. This lack of reliability highlights the necessity for a more intelligent and accurate detection mechanism.

With advancements in Machine Learning and Deep Learning, it is now possible to analyse complex patterns from multi-sensor data and visual inputs. This study leverages these technologies to develop a dual-module system that combines IoT sensor-based detection with image-based fire and smoke recognition. By integrating both approaches, the system improves detection accuracy and reduces false positives compared to traditional methods.

Additionally, the growing demand for smart and safe building environments justifies the development of such a system. The proposed solution provides a scalable and web-based platform that can operate without dedicated hardware, making it practical and cost-effective. It also offers the potential for future integration with real-time IoT devices and surveillance systems, contributing to the advancement of intelligent safety systems.

1.7 Limitations of the Research

Despite achieving high accuracy and effective performance, the proposed system has certain limitations that should be acknowledged. The machine learning module is trained on a specific IoT dataset, and its performance may vary when applied to real-world environments with different sensor configurations or unseen data distributions. The model assumes that input sensor values are accurate and properly calibrated, which may not always be the case in practical deployments.

The deep learning module relies on a relatively limited image dataset for training. Although transfer learning improves performance, the model may face challenges in detecting smoke or fire under extreme conditions such as poor lighting, occlusions, or highly complex backgrounds. Additionally, the YOLOv8 detection model depends on pre-trained weights, and its accuracy is influenced by the quality and diversity of the training data.

Another limitation is that the system currently operates as a simulation-based web application and does not include real-time integration with physical IoT sensors or live camera feeds. While the system is designed to be scalable, deployment in real-world environments would require additional hardware integration, continuous data streaming, and performance optimization for real-time processing.

1.8 Introduction to System Architecture And Functional Modules

The proposed system is designed as a comprehensive, full-stack web application that integrates Machine Learning and Deep Learning techniques for intelligent smoke detection. The architecture follows a modular approach, where different components work together to process input data, perform analysis, and generate accurate predictions. The system is built using the Django framework, which manages user interaction, data processing, and communication between the backend models and the frontend interface. The system architecture consists of two primary modules: the Machine Learning (ML) module and the Deep Learning (DL) module. The ML module processes IoT sensor data containing multiple environmental parameters such as temperature, humidity, gas concentration, and particulate matter. It applies various classification algorithms to predict the presence of smoke based on these inputs. The DL module, on the other hand, focuses on image-based detection using a Convolutional

Neural Network (CNN) with transfer learning, along with a YOLOv8 object detection model for identifying and localising fire and smoke regions within images.

In addition to these core modules, the system includes several functional components such as user authentication, dataset management, model training, and prediction interfaces. Users can interact with the system through a web-based platform to input sensor values or upload images for detection. The modular design ensures scalability, flexibility, and ease of integration with future technologies such as real-time IoT devices and surveillance systems, making the system suitable for smart building safety applications.

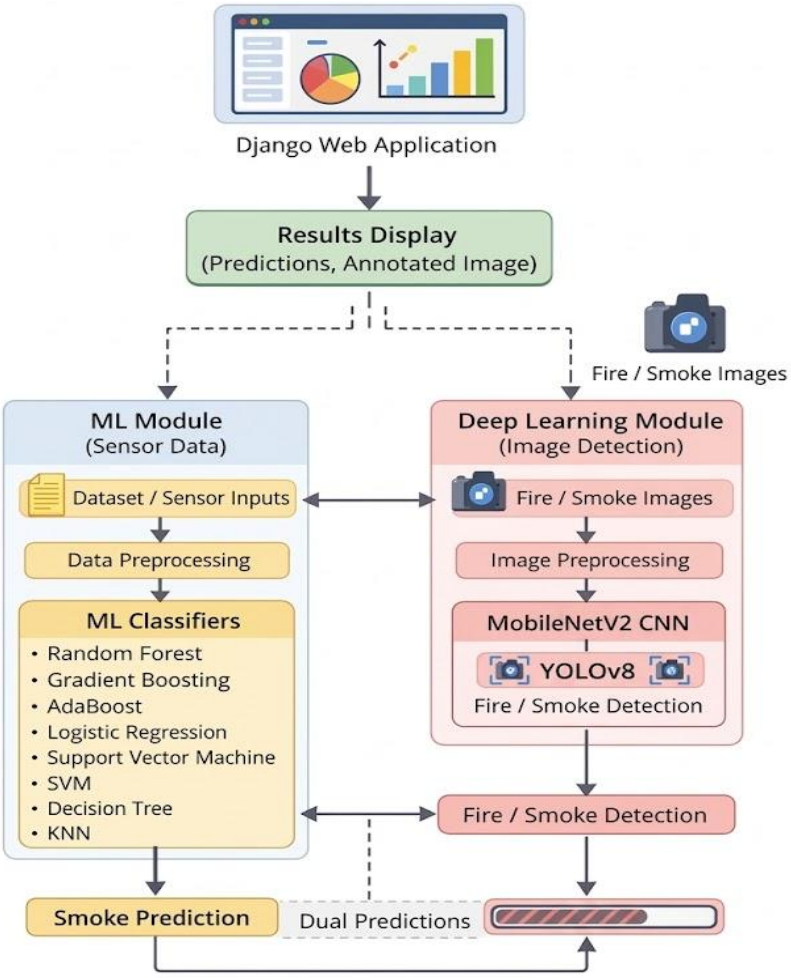

Fig. 1.1: System Architecture Overview – Dual-Module Smoke Detection System

1.9 Evolution of Building Safety in Smoke Detection

The development of smoke detection systems has evolved from basic mechanical detectors to advanced intelligent safety systems. Early smoke detectors were mainly based on ionization and photoelectric sensors, which operated using fixed threshold mechanisms. Although these systems were capable of detecting visible smoke, they often produced false alarms and sometimes failed to detect fires at an early stage due to their reliance on single-sensor readings.

With advancements in technology, modern smoke detection systems began integrating multiple environmental sensors, including temperature, humidity, gas concentration, and particulate matter sensors. This led to the emergence of IoT-based smoke detection systems, where sensor data could be monitored and analyzed in real time. While these systems improved monitoring capabilities, they still depended on predefined rules and lacked the ability to adapt to complex environmental conditions.

Recent developments have introduced Artificial Intelligence (AI) techniques such as Machine Learning and Deep Learning to enhance fire detection systems. Machine learning models analyze data from multiple sensors to identify patterns associated with fire events, improving detection accuracy and reducing false alarms. Deep learning models, particularly Convolutional Neural Networks (CNNs), enable image-based detection of fire and smoke from cameras. In addition, modern object detection models can identify and localize fire regions within images.

These advancements have transformed traditional smoke detection into intelligent fire monitoring systems, enabling faster detection, improved reliability, and better safety management in modern buildings and smart environments.

Smoke detection technologies have progressed from conventional ionization and photoelectric detectors to advanced intelligent monitoring systems. Earlier methods depended on fixed threshold values and single-sensor measurements, which frequently caused delayed responses and false alarms. With the development of IoT, Machine Learning, and Deep Learning, modern systems are capable of analysing multi-sensor environmental data and visual inputs to achieve faster and more reliable fire and smoke detection.

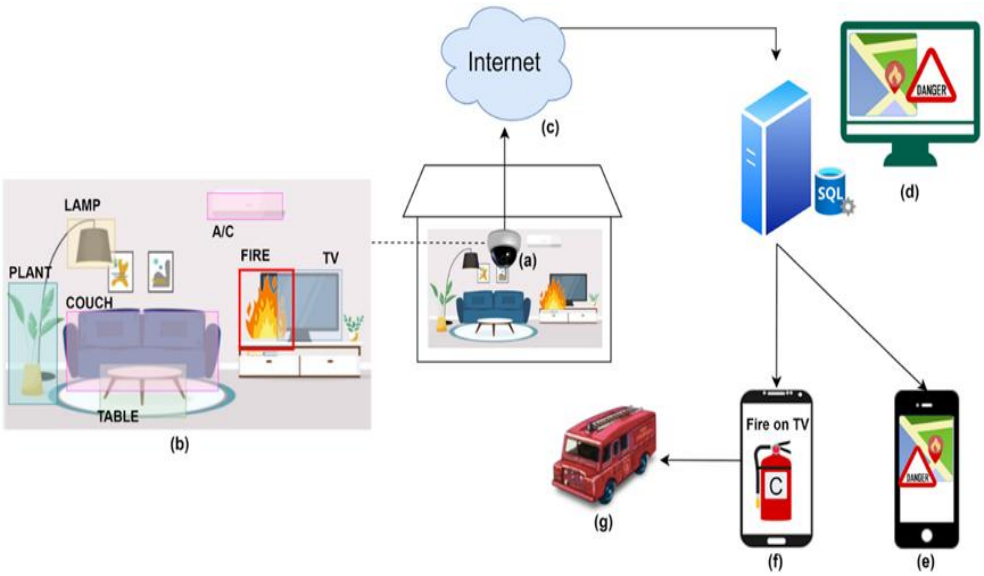

Fig. 1.2: Proposed Fire and Smoke Detection System Using AI and IoT Integration

1.10 Applications of the Proposed System

- Smart buildings for early fire and smoke detection
- Residential apartments and housing complexes for safety monitoring
- Commercial spaces such as malls, offices, and hotels
- Industrial environments and warehouses handling flammable materials
- Smart city surveillance systems integrated with CCTV networks
- Public infrastructure such as airports, railway stations, and hospitals
- Forest fire monitoring using image-based detection systems
- Disaster management and emergency response systems.

The proposed system has wide-ranging applications across various domains where fire safety is critical. In smart buildings and residential environments, it enables early detection of smoke and fire, thereby preventing potential hazards and ensuring occupant safety. In commercial and industrial settings, the system can monitor environmental conditions continuously and reduce the risk of large-scale damage caused by fire incidents. The integration of machine learning and deep learning allows for both sensor-based and visual detection, making it suitable for diverse environments.

Furthermore, the system can be incorporated into smart city infrastructure, where it can work alongside surveillance cameras and IoT networks to provide real-

time safety monitoring. It is also useful in public places such as airports, hospitals, and railway stations where rapid response to fire hazards is essential. Additionally, the image-based detection module can be extended to forest fire monitoring and disaster management systems. Overall, the proposed system provides a scalable, intelligent, and efficient solution for enhancing safety across multiple real-world applications.

1.11 General Working Principle of the Proposed System

The proposed system operates as an intelligent dual-module smoke detection framework that combines machine learning and deep learning techniques to provide accurate and reliable fire detection. The system integrates sensor-based data analysis and image-based detection within a unified web application developed using the Django framework.

The working of the system begins with data acquisition from two different sources. The first source consists of IoT-based environmental sensor data, which includes parameters such as temperature, humidity, gas concentration (TVOC, eCO_2), pressure, and particulate matter levels. The second source consists of image data, where users can upload images or camera-captured frames for visual analysis.

In the machine learning module, the collected sensor data is first preprocessed by removing irrelevant attributes, handling feature scaling using a StandardScaler, and splitting the dataset into training and testing sets. Multiple machine learning algorithms, including Random Forest, Gradient Boosting, Support Vector Machine, K-Nearest Neighbours, Logistic Regression, Decision Tree, and AdaBoost, are trained on this dataset. During prediction, the user inputs real-time sensor values, which are processed and passed through the selected trained model to classify whether smoke or fire is present, along with a confidence score.

In the deep learning module, the system uses a Convolutional Neural Network based on MobileNetV2 with transfer learning. The uploaded image is resized, normalized, and passed through the CNN model to classify it as fire/smoke or non-fire. To enhance detection capability, a YOLOv8 object detection model is integrated, which identifies and localizes fire or smoke regions in the image by drawing bounding boxes with confidence scores.

Both modules are integrated into a web-based interface where users can perform model training, dataset viewing, sensor-based prediction, and image-based detection. The system provides outputs in the form of classification results, confidence levels, and visual annotations, enabling users to make quick and informed decisions.

Overall, the system functions as a comprehensive and intelligent fire detection solution that improves accuracy, reduces false alarms, and enables early warning through the combined use of machine learning, deep learning, and web technologies.

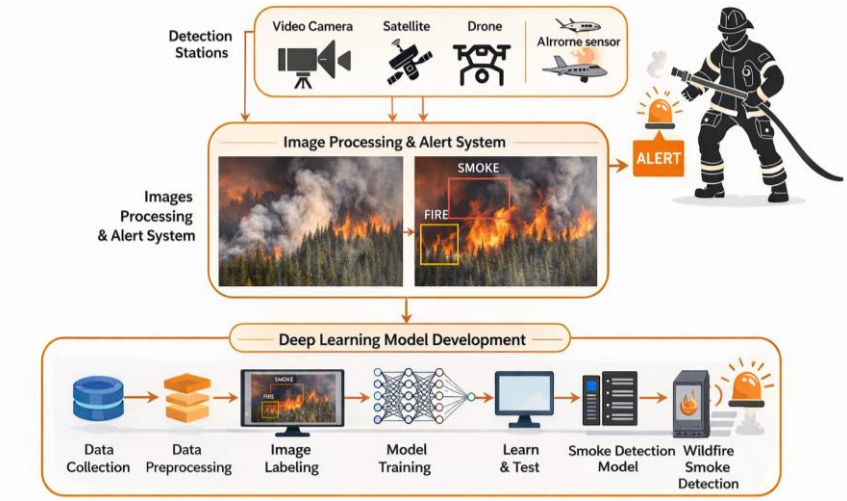

Fig. 1.3: Wildfire smoke-detection operation flowchart using deep learning.

1.12 Organisation of the Report

This report is organised into eight chapters. Chapter 1 provides the introduction, problem statement, and objectives. Chapter 2 presents the literature survey covering related work in IoT-based sensor detection, deep learning for image-based detection, and smart building safety systems. Chapter 3 describes the dataset and data preprocessing. Chapter 4 explains the machine learning module. Chapter 5 explains the deep learning CNN module. Chapter 6 describes the system architecture. Chapter 7 presents the full implementation with sample code and execution workflow. Chapter 8 presents the results, evaluation, original contributions, and conclusion.

CHAPTER 2

LITERATURE REVIEW

2.1 Artificial Intelligence-Based Real-Time Smoke and Fire Detection and Security Management Algorithms (2025)

Author: Khurram Iqbal et al.

Abstract: The rapid advancement of artificial intelligence and deep learning technologies has significantly improved the development of intelligent monitoring systems for safety-critical applications. Fire accidents in residential, commercial, and industrial environments can cause severe damage to property and threaten human lives if not detected at an early stage. This research presents an artificial intelligence-based real-time smoke and fire detection system that utilizes deep learning and computer vision techniques to improve the speed and accuracy of fire detection. The proposed system employs surveillance cameras to continuously monitor the environment and analyze video frames for signs of fire and smoke using advanced object detection algorithms.

Unlike traditional fire detection systems that rely primarily on sensors such as heat, gas, and smoke detectors, the proposed approach leverages visual data for early fire identification. Conventional sensor-based systems are often affected by environmental factors such as dust, sunlight, and gases, which may lead to false alarms, delayed detection, and increased maintenance costs. Additionally, these systems are limited in their ability to monitor large areas in real time. By incorporating artificial intelligence and image processing techniques, the proposed system enhances the capability to detect fire and smoke visually, even in complex environments.

The system integrates Internet of Things (IoT) connectivity and cloud-based communication to enable real-time monitoring and alert generation. Video data captured from CCTV cameras is processed using deep learning models such as Convolutional Neural Networks (CNN) and the YOLO object detection algorithm to identify fire and smoke patterns accurately. When a potential fire is detected, the system automatically sends alerts to users through a cloud-based mobile application, enabling faster response and improved safety management. The proposed approach demonstrates improved detection reliability, reduced false alarms, and enhanced monitoring

capabilities, making it suitable for deployment in modern intelligent surveillance and fire safety systems.

Objectives: The primary objective of this research is to develop an artificial intelligence-based real-time smoke and fire detection system that utilizes deep learning and computer vision techniques to improve early fire detection. The study aims to design an intelligent monitoring framework capable of analyzing real-time video streams captured through surveillance cameras to identify fire and smoke patterns accurately. By integrating advanced object detection algorithms such as YOLO with Convolutional Neural Networks (CNN), the system focuses on enhancing detection reliability while minimizing false alarms caused by environmental disturbances.

Another objective of the research is to incorporate Internet of Things (IoT) connectivity and cloud-based communication to enable continuous monitoring and automated alert generation. The system aims to send real-time notifications to users through mobile applications, allowing faster response to potential fire hazards. Additionally, the study seeks to improve safety management in residential, commercial, and industrial environments by developing a scalable and efficient fire detection system that can monitor large areas and provide timely warnings in emergency situations.

Problem Statement: Traditional fire detection systems are limited by their dependence on environmental sensors that are sensitive to external conditions such as dust, sunlight, and gas interference. These systems often fail to provide reliable early detection and real-time monitoring over large areas, highlighting the need for intelligent vision-based detection methods capable of improving safety and reliability.

Future Scope: Future research can extend the proposed system to smart city infrastructure, industrial safety systems, and large-scale surveillance environments. Integration with technologies such as drones, satellite-based monitoring, and advanced IoT communication networks like 5G can further improve detection coverage, speed, and accuracy.

Solution Enhancement: The proposed system can be enhanced by incorporating advanced deep learning models such as YOLOv8 and lightweight detection architectures to improve speed and efficiency. Additionally, integrating edge computing and energy-efficient hardware can support faster real-time deployment. Improvements in detection under challenging conditions such as low light, fog, and occlusions can further strengthen the reliability of the system.

2.2 Real-Time Fire Object Detection System Using Machine Learning (2025)

Author: Venkata Bhargavi Akuthota et al.

Abstract: Fire accidents pose a significant threat to human life, infrastructure, and environmental safety, making early detection systems essential for minimizing damage and improving emergency response. This research presents a real-time fire detection system that utilizes machine learning and deep learning techniques to enhance early fire identification and safety monitoring. The proposed system integrates both environmental sensor data and visual information to detect fire outbreaks at an early stage. By combining sensor-based monitoring with image-based detection methods, the system aims to improve detection accuracy and provide timely alerts that enable faster response during fire emergencies.

Traditional fire detection methods such as infrared sensors, satellite monitoring, and conventional smoke detectors often experience several limitations. These systems may produce delayed responses, high false alarm rates, and difficulties in handling complex environmental conditions. Most traditional systems rely on fixed threshold values or simple rule-based mechanisms, which restrict their ability to adapt to dynamic environmental variations. Furthermore, conventional detection models often struggle to process large and diverse datasets and fail to capture complex non-linear relationships within environmental data, reducing their overall effectiveness in real-world applications.

To address these limitations, the proposed system employs a machine learning-based detection framework that utilizes multiple classification algorithms including Decision Tree, Random Forest, Support Vector Machine, Logistic Regression, Naïve Bayes, K-Nearest Neighbours, and XGBoost. Environmental parameters such as temperature, humidity, wind speed, and fire-related indices are used as input features for training predictive models. In addition to sensor-based analysis, deep learning techniques such as Convolutional Neural Networks (CNN) and object detection frameworks including YOLO and Faster R-CNN are applied for image-based fire detection.

Objectives: The primary objective of this research is to develop a real-time fire detection system using machine learning and deep learning techniques to enhance early detection of fire outbreaks. The study aims to design an intelligent monitoring framework capable of analyzing environmental sensor data and visual inputs

simultaneously to improve detection accuracy and reduce response time. By integrating multiple machine learning algorithms and deep learning models, the system focuses on identifying fire-related patterns in complex environmental conditions.

Another objective of the research is to improve the reliability and scalability of fire monitoring systems by incorporating advanced data-driven approaches. The study also aims to evaluate the performance of different machine learning classifiers in predicting fire occurrences and identify the most effective algorithm for accurate detection. Additionally, the research seeks to develop a system capable of providing real-time alerts and monitoring capabilities to support safety management in residential, industrial, and environmental monitoring applications.

Problem Statement: Traditional fire detection systems rely heavily on basic sensors and threshold-based mechanisms that often fail to provide accurate and timely detection in complex environments. These systems are prone to false alarms and are limited in their ability to analyze multiple environmental parameters simultaneously. The inability of conventional systems to process large-scale data and identify complex patterns reduces their effectiveness in detecting fires at an early stage, creating a need for more intelligent and data-driven fire detection solutions.

Future Scope: Future research can focus on expanding the proposed system for large-scale applications such as smart cities, forest fire monitoring, and industrial safety systems. Integration with IoT devices, drones, and satellite-based monitoring technologies can further enhance real-time surveillance and detection capabilities. Additionally, incorporating advanced deep learning models and real-time video analytics can improve the efficiency and scalability of fire monitoring systems.

Solution Enhancement: The proposed system can be further enhanced by incorporating larger and more diverse datasets to improve the accuracy and generalization of machine learning models. Integration of advanced deep learning architectures and hybrid detection frameworks can enhance real-time performance and reduce false alarms. Additionally, the use of edge computing and optimized algorithms can improve processing speed, enabling efficient deployment of the system in real-world fire detection and safety management applications.

2.3 A Review on Fire and Smoke Detection with Intelligent Control for Enhanced Safety Using Machine Learning (ML) and Internet of Things (IoT) (2024)

Author: Amey G. Medewar et al.

Abstract: Fire accidents pose serious threats to human life, infrastructure, and the environment, making early detection and rapid response crucial for minimizing damage and improving safety. This research provides a comprehensive review of intelligent fire and smoke detection systems that integrate Machine Learning (ML) and Internet of Things (IoT) technologies to enhance safety and improve firefighting efficiency. Traditional fire detection and firefighting systems mainly depend on manual monitoring or basic automated mechanisms that operate using fixed sensor thresholds. Such systems often experience delayed detection and expose firefighters to dangerous environments involving high temperatures, toxic gases, and poor visibility.

To overcome these limitations, the study explores intelligent monitoring systems capable of detecting fire conditions at an early stage through the integration of smart sensors and advanced data analysis techniques. The proposed approach utilizes IoT sensors to continuously monitor environmental parameters such as temperature, smoke density, and gas concentrations. These sensors collect real-time environmental data that is analyzed using machine learning algorithms including Random Forest and Support Vector Machines to identify patterns associated with fire incidents. Additionally, deep learning techniques such as Convolutional Neural Networks (CNN) are used for image-based fire detection through surveillance cameras.

The research also highlights the use of autonomous firefighting robots capable of operating in hazardous environments where human intervention may be dangerous. These robots can assist in fire suppression and monitoring tasks, thereby improving response efficiency and reducing the risks faced by firefighters. The integration of IoT-based monitoring, machine learning models, and robotic systems enables the development of a more intelligent and efficient fire detection and response system capable of supporting real-time monitoring and decision-making.

Objectives: The primary objective of this research is to review and analyze intelligent fire and smoke detection systems that integrate Machine Learning and Internet of Things technologies to improve safety and firefighting efficiency. The study aims to

evaluate how advanced data-driven approaches can enhance early fire detection by continuously monitoring environmental parameters through IoT-based sensors.

Another objective of the research is to explore the potential of combining sensor-based monitoring, image-based detection, and automated firefighting mechanisms to create an intelligent fire management system. The study also aims to highlight the role of machine learning algorithms in analyzing environmental data to detect fire conditions accurately and support rapid response in hazardous environments.

Problem Statement: Traditional fire detection and firefighting systems rely heavily on manual monitoring and simple sensor-based mechanisms that operate using fixed thresholds. These systems often fail to detect fire at an early stage and may expose firefighters to dangerous conditions such as high temperatures, toxic gases, and poor visibility. The limitations of conventional systems highlight the need for intelligent monitoring solutions that can analyze environmental data in real time and provide faster and more reliable fire detection.

Future Scope: Future research can focus on expanding intelligent fire detection systems for large-scale applications such as smart city infrastructure, industrial safety monitoring, and disaster management systems. Integration with emerging technologies such as drones, edge computing, and advanced IoT communication networks can further enhance detection speed, coverage, and monitoring efficiency.

Solution Enhancement: The proposed system can be enhanced by incorporating advanced deep learning models such as YOLO and multimodal learning techniques to improve fire detection accuracy. Additionally, reinforcement learning can be used to support autonomous decision-making in firefighting robots. Integration of larger datasets, real-time analytics, and improved sensor networks can further strengthen the reliability and scalability of intelligent fire detection systems in real-world applications. Furthermore, implementing edge computing techniques can reduce latency and enable faster real-time processing of sensor data. The use of advanced data fusion methods to combine information from multiple sensors and cameras can also improve detection accuracy and reduce false alarms. These improvements can significantly enhance the overall efficiency, adaptability, and responsiveness of intelligent fire monitoring systems.

2.4 Enhancing Fire Detection in Smart Buildings Using Deep Learning and IoT (2023)

Author: Ahmed Alshammari et al.

Abstract: The rapid growth of smart building technologies has increased the demand for intelligent safety systems capable of detecting fire and smoke incidents at an early stage. This study focuses on developing an advanced fire and smoke detection system for smart buildings by integrating deep learning techniques with Internet of Things (IoT) based sensor networks. Traditional smoke detection systems mainly rely on single sensor readings and fixed threshold mechanisms, which often fail to detect fire accurately in complex environments. These systems are also limited in their ability to identify smoke during the early stages of fire outbreaks and often produce false alarms due to environmental factors.

To address these limitations, the research proposes an intelligent fire detection framework that combines environmental sensor data with image-based analysis. The proposed system integrates IoT sensors and deep learning models to continuously monitor environmental conditions in real time. Sensors collect important parameters such as temperature, humidity, and gas concentration, while cameras capture real-time images of the surrounding environment. These data sources provide complementary information that helps improve the accuracy of fire detection systems.

A Convolutional Neural Network (CNN) model is used for image-based smoke detection, while machine learning algorithms analyze sensor data to identify abnormal environmental patterns associated with fire conditions. The outputs from both sensor-based and image-based analysis are combined to enhance the reliability and robustness of the detection process. This hybrid detection approach improves early fire detection accuracy and enables faster response compared to traditional systems. The integration of IoT connectivity also allows continuous monitoring and real-time data transmission, making the system highly suitable for deployment in modern smart building environments.

Objectives: The primary objective of this research is to develop an intelligent fire and smoke detection system for smart buildings by integrating deep learning techniques with IoT-based sensor networks. The study aims to improve early fire detection accuracy by combining sensor data and image-based analysis to identify fire-related patterns more effectively.

Another objective of the research is to design a real-time monitoring system capable of analyzing environmental parameters such as temperature, humidity, and gas concentration while simultaneously processing visual information captured by cameras. By combining multiple data sources, the system seeks to enhance detection reliability and reduce false alarms in complex building environments.

Problem Statement: Traditional fire detection systems in buildings often rely on single sensor inputs and fixed threshold values, which can result in delayed detection or false alarms under varying environmental conditions. These systems lack the capability to process multiple data sources simultaneously and are not well suited for complex environments found in modern smart buildings. Therefore, there is a need for intelligent fire detection systems that can integrate sensor networks with deep learning models to provide faster and more accurate detection.

Future Scope: Future research can focus on expanding the proposed system for large-scale monitoring environments such as smart cities, industrial facilities, and public infrastructure. Integration with automated emergency response systems including alarms, sprinkler systems, and evacuation management platforms can further enhance building safety and emergency response capabilities. The system can also be integrated with advanced IoT communication networks to support faster data transmission and real-time monitoring across multiple locations. Additionally, incorporating drone-based surveillance and remote monitoring systems can help detect fire incidents in large or hard-to-reach areas. Further developments may also include predictive analytics and intelligent risk assessment models that can analyze environmental patterns to predict potential fire hazards before they occur, thereby improving preventive safety measures in smart building infrastructures.

Solution Enhancement: The proposed system can be further enhanced by incorporating advanced deep learning models such as Vision Transformers (ViT) to improve image analysis and fire detection accuracy. The use of edge AI devices can enable faster real-time processing and reduce dependency on centralized computing systems. Additionally, integrating cloud-based platforms for large-scale data storage and analysis can improve scalability and continuous learning of the detection models. Further improvements such as multimodal data fusion, improved sensor networks, and optimized deep learning architectures can significantly enhance the reliability and efficiency of intelligent fire detection systems in smart building environments.

2.5 Enhancing Building Safety Through Machine Learning-Based Smoke Detection (2022)

Author: Sulaiman Syed Mohamed et al.

Abstract: Ensuring early smoke detection is essential for preventing fire accidents and improving building safety in residential, commercial, and industrial environments. This research focuses on developing an intelligent smoke detection system using machine learning techniques applied to multi-sensor IoT data. Traditional smoke detection systems such as ionization and photoelectric detectors rely heavily on fixed threshold values to identify smoke presence. However, these systems often produce false alarms due to environmental factors such as dust, humidity, and aerosols, and they may fail to accurately detect smoke during the early stages of fire incidents.

In addition, conventional smoke detection systems lack adaptability to changing environmental conditions and are unable to analyze multiple environmental parameters simultaneously. These limitations reduce their effectiveness in complex environments where different atmospheric conditions may influence sensor readings. To overcome these challenges, the study proposes a machine learning-based approach that can analyze complex patterns in environmental data collected from multiple sensors, thereby improving the accuracy and reliability of smoke detection systems.

The proposed system utilizes real-world IoT sensor data collected from environmental sensors measuring parameters such as temperature, humidity, total volatile organic compounds (TVOC), hydrogen, ethanol, and air pressure. Several supervised machine learning algorithms including Naive Bayes, Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Gradient Boosting, and AdaBoost are applied to analyze the sensor data and detect smoke-related conditions. Experimental evaluation using performance metrics such as accuracy and recall shows that ensemble learning techniques including Random Forest, Decision Tree, Gradient Boosting, and AdaBoost achieve high accuracy and demonstrate improved performance compared to traditional smoke detection methods.

Objectives: The primary objective of this research is to develop an intelligent smoke detection system using machine learning techniques and multi-sensor IoT data to improve building safety. The study aims to analyze environmental sensor data collected from different sources and identify patterns associated with smoke or fire-related conditions.

Another objective of the research is to evaluate the performance of various supervised machine learning algorithms in detecting smoke events and determine the most effective model for accurate detection. The system also aims to support continuous environmental monitoring through IoT integration, enabling real-time detection and improved reliability in safety monitoring systems.

Problem Statement: Traditional smoke detection systems rely on fixed threshold mechanisms and single sensor readings, which often lead to false alarms or delayed detection under varying environmental conditions. These systems are unable to analyze multiple environmental parameters simultaneously and lack adaptability to dynamic atmospheric conditions, creating a need for intelligent detection systems capable of analyzing complex multi-sensor data for accurate smoke detection.

Future Scope: Future research can focus on expanding the proposed smoke detection system for deployment in smart buildings, residential complexes, and industrial safety environments where continuous monitoring is required. The system can also be extended to large-scale IoT-based safety infrastructures such as smart cities to enable automated monitoring and alert generation. Further developments may include integrating advanced deep learning models such as Convolutional Neural Networks (CNN) and Vision Transformers (ViT) for image and video-based smoke detection. Additionally, incorporating real-time analytics, edge computing, and advanced sensor networks can further enhance system performance and scalability in modern intelligent safety systems.

Solution Enhancement: The proposed system can be enhanced by integrating hybrid machine learning and deep learning architectures to improve detection accuracy and adaptability. Utilizing larger and more diverse datasets can help improve model generalization and robustness in different environmental conditions. Additionally, combining sensor-based analysis with image-based smoke detection techniques can further strengthen detection reliability. The use of optimized algorithms, improved feature selection methods, and real-time IoT data processing can significantly enhance the efficiency and responsiveness of intelligent smoke detection systems for real-world deployment.

CHAPTER 3

DATASET AND DATA PREPROCESSING

3.1 ML Module Dataset

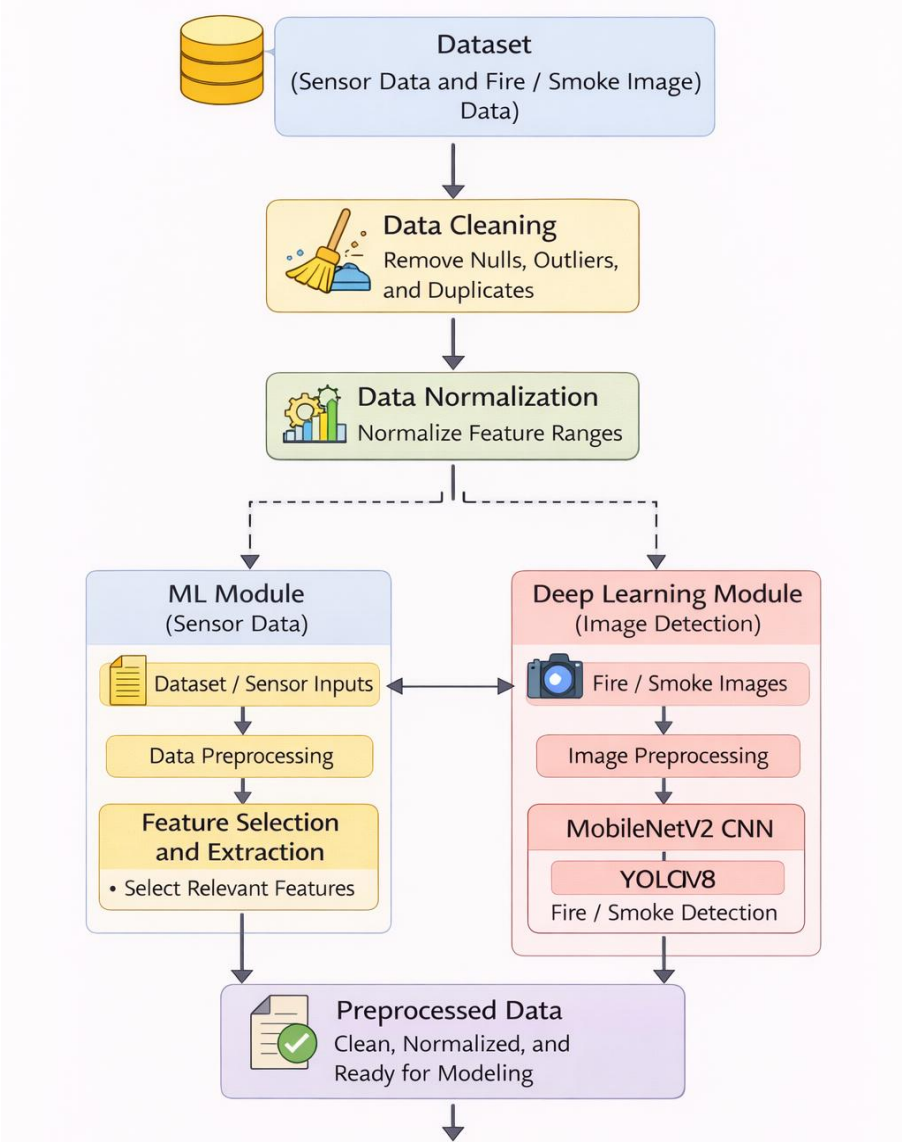

Fig. 3.1: Dataset Preprocessing Pipeline Flowchart

The machine learning module uses the smoke_detection_iot.csv dataset, a publicly available real-world IoT sensor dataset collected from environmental sensors installed in a building. The dataset was collected to study the relationship between various environmental measurements and the presence of a fire alarm condition.

3.1.1 Dataset Summary

Table 3.1: IoT Sensor Dataset Summary

Property	Value
Dataset Name	smoke_detection_iot.csv
Total Records	62,630 rows
Number of Feature Columns	13 input features
Target Column	Fire Alarm (0 = No Smoke, 1 = Smoke Detected)
Source	Real IoT sensor readings from a building environment
File Format	CSV (Comma-Separated Values)

3.1.2 Feature Columns

The dataset used for the smoke detection system contains 15 columns, including a serial number (S.No), a UTC timestamp, 13 environmental sensor feature columns, and the target column **Fire Alarm**. The serial number and timestamp serve as identification and time-tracking fields, while the 13 sensor features are used as input variables for training the machine learning models. The **Fire Alarm** column acts as the target variable, where a value of 1 indicates the presence of smoke or fire and 0 represents normal environmental conditions. The detailed description of these input features is presented in **Table 3.2**.

The sensor features represent various environmental and air-quality parameters collected from an IoT-based monitoring system. These include temperature, humidity, gas concentrations such as **TVOC** and **eCO_2**, raw hydrogen and ethanol sensor readings, atmospheric pressure, and particulate matter measurements like **PM1.0** and **PM2.5**. Additional features such as **NC0.5, NC1.0, and NC2.5** represent particle concentration levels of different sizes, while the **CNT** feature records the number of sensor samples collected over time. Together, these features provide important environmental information that enables machine learning models to analyze patterns and accurately detect smoke or fire-related conditions.

Table 3.2: Input Feature Columns

Feature Name	Description	Unit
Temperature[C]	Ambient temperature of the environment	Degrees Celsius
Humidity [%]	Relative humidity of the air	Percentage
TVOC [ppb]	Total Volatile Organic Compounds concentration	Parts Per Billion
eCO2 [ppm]	Equivalent Carbon Dioxide concentration	Parts Per Million
Raw H2	Raw hydrogen gas sensor reading	Sensor Units
Raw Ethanol	Raw ethanol vapour sensor reading	Sensor Units
Pressure[hPa]	Atmospheric pressure	Hectopascals
PM1.0	Particulate matter concentration, 1.0 micron	Micrograms per m3
PM2.5	Particulate matter concentration, 2.5 microns	Micrograms per m3
NC0.5	Number concentration of particles at 0.5 microns	Particles per cm3
NC1.0	Number concentration of particles at 1.0 micron	Particles per cm3
NC2.5	Number concentration of particles at 2.5 microns	Particles per cm3
CNT	Running count of sensor samples collected	Count

The target column is Fire Alarm, a binary label where 1 indicates a fire alarm condition and 0 indicates normal environmental conditions.

3.1.3 Class Distribution Analysis

Analysis of the target column reveals a moderate class imbalance in the dataset. Out of 62,630 total records, 44,757 samples (71.46%) carry a Fire Alarm label of 1 (smoke/fire detected), while 17,873 samples (28.54%) carry a label of 0 (no smoke). This approximately 71:29 ratio indicates that the majority class is fire-alarm-positive, which reflects the real-world sensor collection methodology where fire scenarios were deliberately over-represented to ensure sufficient positive training examples.

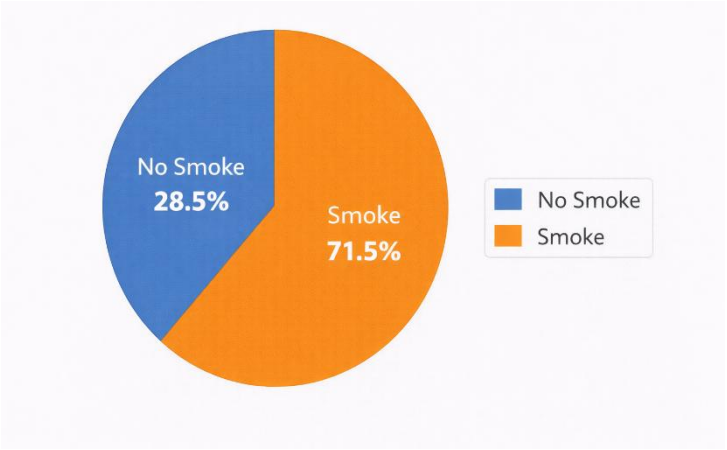

Fig. 3.2: Fire Alarm Class Distribution

This class imbalance has two important implications for model evaluation. First, raw accuracy alone is not a reliable metric — a naive classifier predicting all samples as Fire Alarm = 1 would achieve 71.46% accuracy without any learning. This justifies the use of AUC-ROC, Precision, Recall, and IoU as the primary evaluation metrics in this project, since these metrics assess discriminative ability independently of class distribution. Second, the fact that all seven trained classifiers nonetheless achieved AUC-ROC scores above 0.999 confirms that the 13 sensor channels provide a strongly separable signal between fire and non-fire environmental states, and that the imbalance does not introduce bias into the learnt decision boundaries when the input features are sufficiently discriminative.

3.2 Data Preprocessing

Before training the machine learning models, the dataset undergoes the following preprocessing steps:

- **Step 1 – Load Dataset**: The CSV file is loaded from the media directory using Pandas into a DataFrame.
- **Step 2 – Drop Unnecessary Columns**: The S.No and UTC columns are dropped as they are identifiers not useful for classification.
- **Step 3 – Define Features and Target**: The 13 sensor columns form the feature matrix X. The Fire Alarm column is the target vector y.
- **Step 4 – Train-Test Split**: The dataset is split 80/20 using scikit-learn's train_test_split with random_state=42.
- **Step 5 – Feature Scaling**: A StandardScaler is fitted on the training set and applied to both sets. The scaler is saved as scaler.pkl.

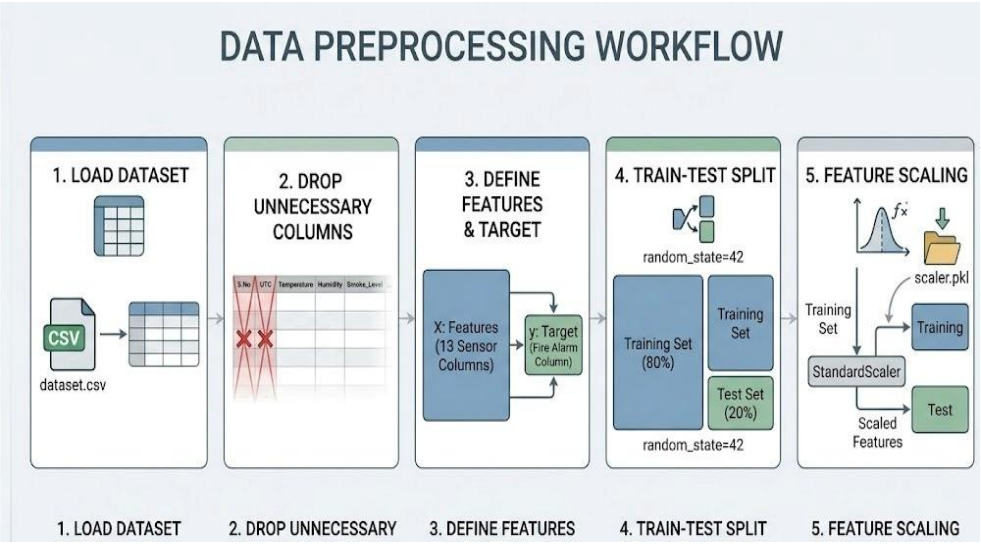

Fig. 3.3: Data Preprocessing Workflow for Smoke Detection System

3.3 CNN Module Dataset

The deep learning module uses a fire and smoke image dataset sourced from Kaggle (phylake1337), organised into fire and no_fire folders inside media/cnn_dataset/.

Table 3.3: CNN Image Dataset Summary

Property	Value
Dataset Source	Kaggle Fire Dataset (phylake1337)
Fire / Smoke Images	755 images
Non-Fire Images	244 images
Total Images	998 images (799 train + 199 validation)
Training Split	80 percent (799 images)
Validation Split	20 percent (199 images)
Image Format	JPG and PNG
Input Size to CNN	224 × 224 pixels, 3 colour channels (RGB)

Before images are fed into the CNN model, they are resized to 224 × 224 pixels and converted to RGB format. Data augmentation including horizontal flipping, zoom, and rotation is applied using Keras ImageDataGenerator.

To further enhance the robustness of the CNN model, normalization is applied to the image pixel values by scaling them to a range between 0 and 1. This ensures faster convergence during training and prevents issues related to varying illumination conditions across different images. Additionally, the dataset is shuffled during training to reduce bias and ensure that the model generalizes well across both fire and non-fire classes. The use of transfer learning with MobileNetV2 allows the model to leverage pre-trained feature extraction capabilities, significantly improving performance even with a relatively small dataset.

Furthermore, class balance is carefully maintained to prevent the model from becoming biased toward the majority class. Although the dataset contains more fire images than non-fire images, augmentation techniques help in artificially increasing the diversity of the minority class. The training process is monitored using validation metrics to avoid overfitting, and early stopping techniques can be incorporated in future improvements.

CHAPTER 4

ML SYSTEM WORKFLOW

4.1 Introduction to the ML Module

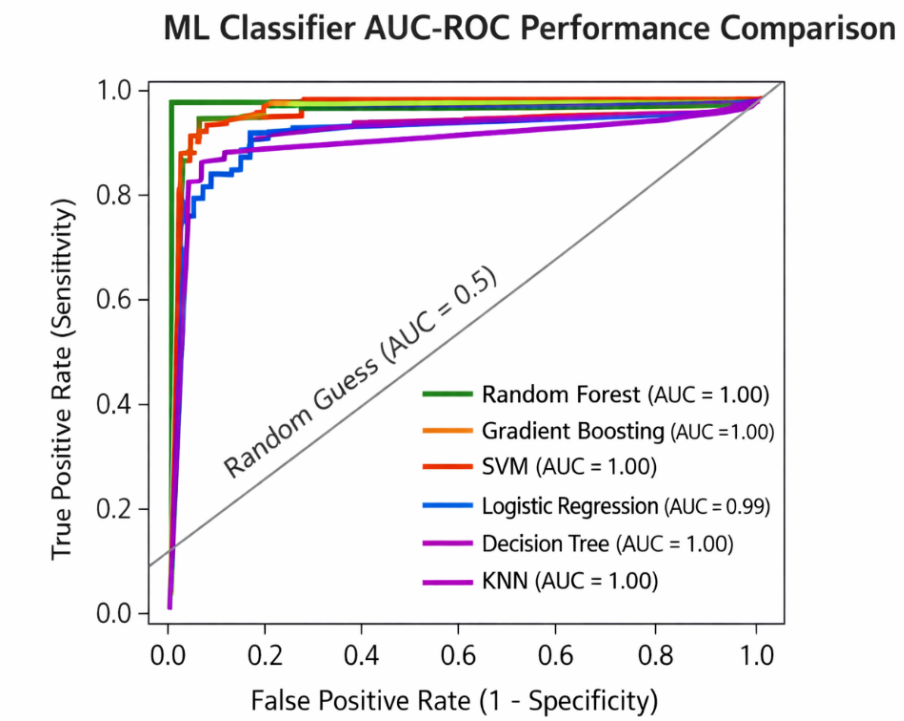

Fig. 4.1: ML Classifier AUC-ROC Performance Comparison

The machine learning module analyses environmental sensor readings from IoT devices and classifies whether they correspond to a smoke or fire condition. Seven different machine learning classifiers are trained and evaluated on the IoT dataset, and the user can select any model to make a prediction through the web interface.

4.2 Algorithms Used

4.2.1 Random Forest Classifier

Random Forest is an ensemble method that constructs multiple decision trees and outputs the majority class. It is highly effective for tabular datasets and resistant to overfitting. It was selected as the primary model due to its consistently high accuracy.

4.2.2 Gradient Boosting Classifier

Gradient Boosting builds an additive model stage-wise, fitting each new tree to correct residual errors of the previous ensemble. It is one of the most powerful algorithms for structured data and achieves near-perfect accuracy by detecting complex non-linear patterns.

4.2.3 AdaBoost Classifier

AdaBoost combines multiple weak classifiers into a strong classifier by iteratively training each on samples misclassified by the previous ensemble. It adaptively reweights training samples and performs comparably to the ensemble tree methods.

4.2.4 Logistic Regression

Logistic Regression models the probability of a binary outcome using a sigmoid function applied to a linear combination of input features. It provides interpretable coefficients, fast training, and achieves high accuracy on this dataset.

4.2.5 Support Vector Machine (SVM)

SVM with a Radial Basis Function kernel finds the optimal hyperplane that maximises the margin between two classes. SVM with probability=True enables confidence score output and produces a robust classifier on scaled features.

4.2.6 Decision Tree Classifier

A Decision Tree recursively partitions the feature space by selecting the feature and threshold that best separates classes at each node. It produces a highly interpretable structure with high accuracy on the clean dataset patterns.

4.2.7 K-Nearest Neighbours (KNN)

KNN classifies a new data point based on the majority class of its k nearest neighbours using Euclidean distance. It is a non-parametric, instance-based algorithm sensitive to feature scaling, which is why StandardScaler is applied before training and prediction.

4.3 Training Process

All seven classifiers are trained simultaneously when the user clicks Train All Models. The steps of the training process are as follows:

- **Step 1**: The dataset is loaded and preprocessed as described in Chapter 3.
- **Step 2**: For each classifier, the fit method is called on the scaled training data.
- **Step 3**: Each model predicts on the scaled test data to generate predictions and probability estimates.
- **Step 4**: Six metrics are calculated: Precision, Recall, AUC-ROC, Intersection over Union (IoU), Accuracy, and F1-Score.
- **Step 5**: Each trained model is saved as a .pkl file in media/models/ using joblib.dump.
- **Step 6**: The best model by AUC-ROC score is identified and highlighted.
- **Step 7**: All metrics are passed to the template for display as cards, bar charts, and a radar chart.

4.4 Evaluation Metrics

4.4.1 Precision

Precision measures the proportion of positive predictions that are actually correct: True Positives divided by the sum of True Positives and False Positives. Five models (Random Forest, Gradient Boosting, Decision Tree, KNN, and SVM) achieve precision of 99.99%–100%. AdaBoost achieves 99.97% and Logistic Regression achieves 99.31%.

4.4.2 Recall

Recall measures the proportion of actual positive cases correctly identified: True Positives divided by the sum of True Positives and False Negatives. High recall is especially important in a safety application.

4.4.3 AUC-ROC Score

The AUC-ROC measures the model's ability to discriminate between two classes across all probability thresholds. A value of 1.0 represents a perfect classifier. Six models achieve a perfect AUC-ROC of 1.0000. Logistic Regression achieves 0.9989, confirming excellent discriminative ability even for the linear model.

4.4.4 Intersection Over Union (IoU)

IoU for binary classification is calculated as the intersection of predicted positives and actual positives divided by their union. It measures overlap between predicted and actual alarm conditions. IoU is widely used as a performance metric to evaluate the similarity between predicted and actual outcomes in classification and detection tasks.

A higher IoU value indicates a greater overlap between predicted fire alarm conditions and the true fire events in the dataset. This metric helps assess how accurately the model identifies fire-related instances while minimizing false detections

4.5 Model Performance Results

The results presented in Table 4.1 demonstrate that the majority of the machine learning models achieved exceptionally high performance across all evaluation metrics, including Precision, Recall, AUC-ROC, IoU, and F1-Score. Models such as Random Forest, Gradient Boosting, Decision Tree, and KNN achieved perfect scores of 100.00% across all metrics, indicating their ability to accurately distinguish between fire and non-fire conditions in the dataset. These results highlight the presence of strong and well-defined patterns within the IoT sensor data, enabling the models to learn highly reliable decision boundaries.

Among the models, AdaBoost and Logistic Regression exhibited slightly lower performance compared to the others, with Logistic Regression showing the lowest scores, particularly in IoU (98.75%) and F1-Score (99.37%). Despite this, their performance remains highly competitive and suitable for practical applications. The near-perfect results across multiple models suggest that ensemble and tree-based algorithms are particularly effective for this classification task. Overall, the

performance analysis confirms the robustness and accuracy of the proposed machine learning approach for smoke detection in building safety systems.

Table 4.1: ML Model Performance Results

Model	Precision	Recall	AUC-ROC	IoU	F1-Score
Random Forest	100.00%	100.00%	100.00%	100.00%	100.00%
Gradient Boosting	100.00%	100.00%	100.00%	100.00%	100.00%
AdaBoost	99.97%	99.97%	100.00%	99.93%	99.97%
Logistic Regression	99.31%	99.44%	99.89%	98.75%	99.37%
SVM	99.99%	100.00%	100.00%	99.99%	99.99%
Decision Tree	100.00%	100.00%	100.00%	100.00%	100.00%
KNN	100.00%	100.00%	100.00%	100.00%	100.00%

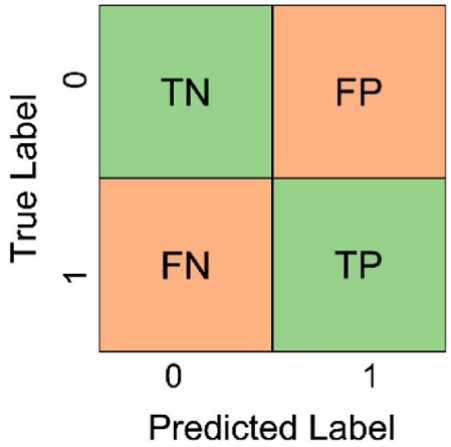

Fig. 4.2: Confusion Matrix of Random Forest Classifier

The confusion matrix evaluates the performance of the Random Forest model by comparing predicted labels with actual labels. True positives indicate correctly detected fire conditions, while true negatives represent correctly identified non-fire cases. The very low number of false positives and false negatives demonstrates the model's high accuracy and strong ability to distinguish between fire and non-fire conditions, improving the reliability of the detection system.

4.6 5-Fold Cross-Validation

To validate that classifier performance is consistent and not an artefact of a single train-test split, 5-fold cross-validation was performed on three representative classifiers — Random Forest, Gradient Boosting, and SVM — using AUC-ROC as the scoring metric. The full dataset is partitioned into five equal folds; in each iteration one fold is held out for evaluation and the remaining four are used for training. The results confirm that all three classifiers maintain near-perfect AUC-ROC across every fold with negligible variance, providing strong evidence of robust generalisation and the absence of overfitting.

Table 4.2: 5-Fold Cross-Validation AUC-ROC Results (score per fold and mean)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean
Random Forest	1.0000	1.0000	1.0000	1.0000	1.0000	**1.0000**
Gradient Boosting	1.0000	1.0000	0.9999	1.0000	1.0000	**1.0000**
SVM	1.0000	1.0000	0.9998	0.9999	0.9999	**0.9999**

4.7 Prediction Workflow

When a user submits the sensor prediction form, the following steps are executed:

- **Step 1**: The 13 sensor values are validated and converted to float.
- **Step 2**: The selected model's .pkl file is loaded from media/models/ using joblib.load.
- **Step 3**: scaler.pkl is loaded and used to scale the input values.
- **Step 4**: The scaled input is passed to the model's predict and predict_proba methods.
- **Step 5**: The binary prediction and confidence percentage are returned and displayed to the user.

CHAPTER 5

DEEP LEARNING MODULE

5.1 Introduction to the DL Module

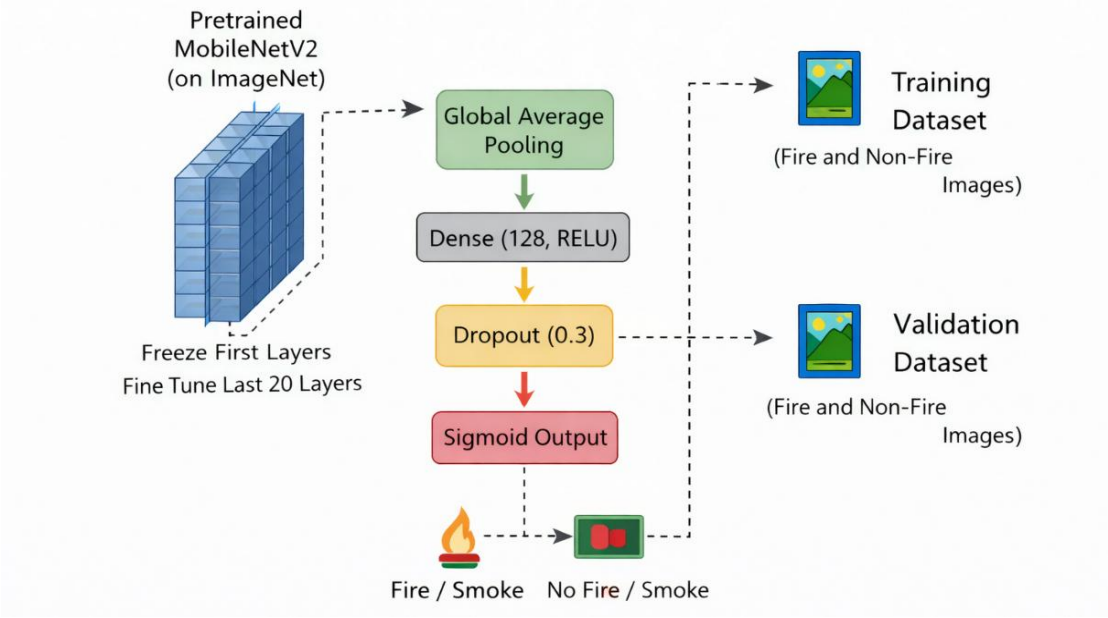

Fig. 5.1: MobileNetV2 Transfer Learning Architecture for Fire/Smoke Detection

The deep learning module accepts an image uploaded by the user and uses a trained convolutional neural network to classify the image as fire (fire or smoke present) or no fire (normal scene). This visual detection capability complements the sensor-based ML module and provides an alternative pathway for detection when camera feeds or photographs are available.

5.2 Convolutional Neural Networks

A Convolutional Neural Network (CNN) is a class of deep neural network specifically designed for processing grid-structured data such as images. A CNN learns spatial hierarchies of features by applying learnable filters across the input. Early layers detect simple patterns such as edges and textures, while deeper layers combine these into object-level representations.

CNNs have become the dominant approach for image classification tasks due to their ability to automatically learn relevant features directly from pixel data. The

architecture consists of alternating convolutional and pooling layers followed by fully connected layers and a final classification layer.

5.3 Transfer Learning with Mobilenetv2

Training a deep CNN from scratch requires a large labelled dataset and significant computational resources. This project uses transfer learning, leveraging a pre-trained MobileNetV2 network trained on ImageNet (1.4 million images, 1,000 classes) as the starting point.

MobileNetV2 was selected due to its efficient design using depthwise separable convolutions, which significantly reduce the number of parameters while maintaining competitive accuracy. The model has approximately 3.4 million parameters and is optimised for inference speed, making it suitable for deployment in web applications.

5.4 Model Architecture

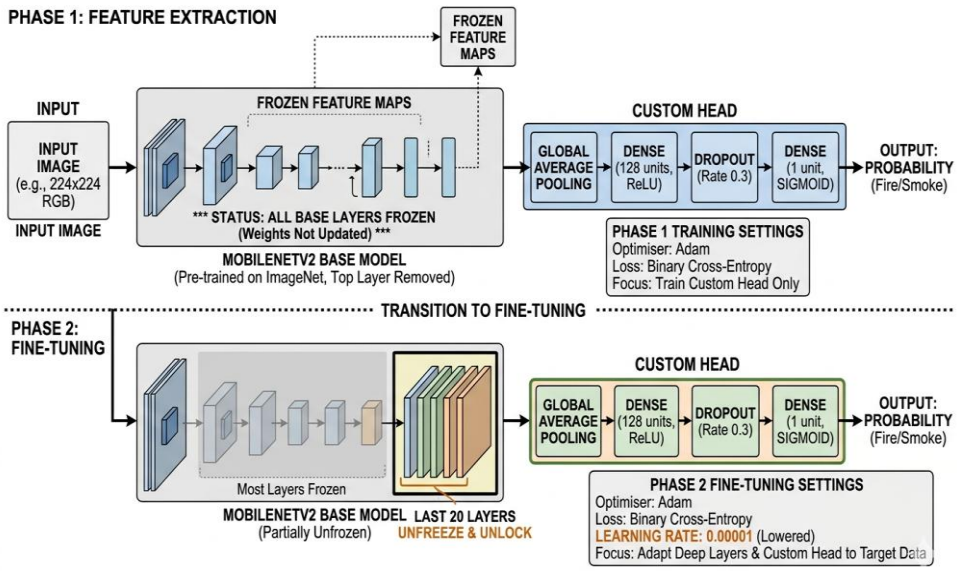

Fig. 5.2: Two-Phase CNN Architecture Flow for Fire using MobileNetV2

The CNN model is constructed in two phases. In Phase 1, the MobileNetV2 base model (pre-trained on ImageNet, top layer removed) is loaded with all base layers frozen. A custom head consisting of global average pooling, a 128-unit dense layer with ReLU, a dropout layer (rate 0.3), and a sigmoid output neuron is appended. The model is compiled with Adam optimiser and binary cross-entropy loss.

In Phase 2, fine-tuning is performed by unfreezing the last 20 layers of MobileNetV2 and retraining with a lower learning rate of 0.00001, allowing the deep layers to adapt to the specific visual features of fire and smoke.

5.5 Training Process

The CNN model is trained using the train_cnn.py script. Keras ImageDataGenerator is used for data augmentation and automatic class labelling from the folder structure.

Table 5.1: CNN Training Configuration

Parameter	Value
Input Image Size	224 × 224 pixels, RGB
Batch Size	32
Phase 1 Epochs	10 (frozen base)
Phase 2 Epochs	10 (fine-tuning last 20 layers)
Phase 1 Learning Rate	0.001 (Adam)
Phase 2 Learning Rate	0.00001 (Adam)
Loss Function	Binary Cross-Entropy
Callbacks	ModelCheckpoint (best val_accuracy), EarlyStopping (patience=3)
Data Augmentation	Horizontal flip, zoom range 0.2, rotation range 20°
Validation Split	20 percent (automatic via ImageDataGenerator)

The training process uses data augmentation techniques such as flipping, zooming, and rotation to increase dataset diversity and improve the model's ability to generalize to real-world fire and smoke conditions.

5.6 Training Results

Table 5.2: CNN Training Results

Metric	Value
Best Validation Accuracy	96.98 percent
Best Epoch	Epoch 10
Phase 1 Final val_accuracy	~92 percent
Phase 2 Best val_accuracy	96.98 percent
Average GPU Epoch Time	~13 seconds
Average CPU Epoch Time	~30 seconds
Model Saved As	media/cnn_model.h5
Classes	fire (index 0), no_fire (index 1)

The model training completed 10 epochs in Phase 1 and 10 epochs in Phase 2. The best validation accuracy of 96.98 percent was recorded at epoch 10. GPU training on the NVIDIA GeForce RTX 3050 Ti reduced per-epoch time from approximately 30 seconds (CPU) to approximately 13 seconds, a 2.3x speedup.

The training results presented in Table 5.2 indicate that the CNN model achieved a high level of performance, with a best validation accuracy of 96.98 percent recorded at epoch 10 during Phase 2 fine-tuning. The improvement from approximately 92 percent validation accuracy in Phase 1 to 96.98 percent in Phase 2 highlights the effectiveness of the fine-tuning process, where the deeper layers of MobileNetV2 are allowed to adapt to domain-specific features related to fire and smoke. This demonstrates that transfer learning combined with selective layer unfreezing significantly enhances model performance.

In addition to accuracy improvements, the training process also showed substantial gains in computational efficiency through GPU acceleration. The average

epoch time was reduced from approximately 30 seconds on CPU to around 13 seconds on GPU, resulting in a 2.3× speedup. This reduction in training time makes the model more practical for iterative development and real-world deployment scenarios. Overall, the training results confirm that the proposed CNN model is both accurate and efficient, making it suitable for reliable fire detection applications.

5.7 Prediction Workflow

When a user uploads an image, the CNNPrediction view runs two detection passes on the same image: CNN classification followed by YOLO fire and smoke detection. The steps are as follows:

- **Step 1**: The uploaded image file is saved permanently to media/uploads/ with a UUID filename.
- **Step 2**: The image is preprocessed to 224 x 224 RGB, normalised by dividing by 255, and passed to cnn_model.h5 for CNN classification.
- **Step 3**: The sigmoid output is interpreted: since fire has class index 0, fire probability is 1 minus the sigmoid value.
- **Step 4**: The CNN result (SMOKE / FIRE DETECTED or NO SMOKE DETECTED) and confidence percentage are prepared for display.
- **Step 5**: If media/models/best.pt exists, the YOLOv8 model is loaded and run on the same saved image with a confidence threshold of 0.25.
- **Step 6**: For each YOLO detection, a bounding box is drawn on the image using OpenCV with a colour-coded label (e.g. Fire 0.56, Smoke 0.81).
- **Step 7**: The annotated image is saved to media/results/ and its URL is passed to the template.
- **Step 8**: The template displays both: the CNN text result with confidence bar in the top panel, and the YOLO annotated image with a detection log below.

CHAPTER 6

SYSTEM DESIGN AND ARCHITECTURE

6.1 System Overview

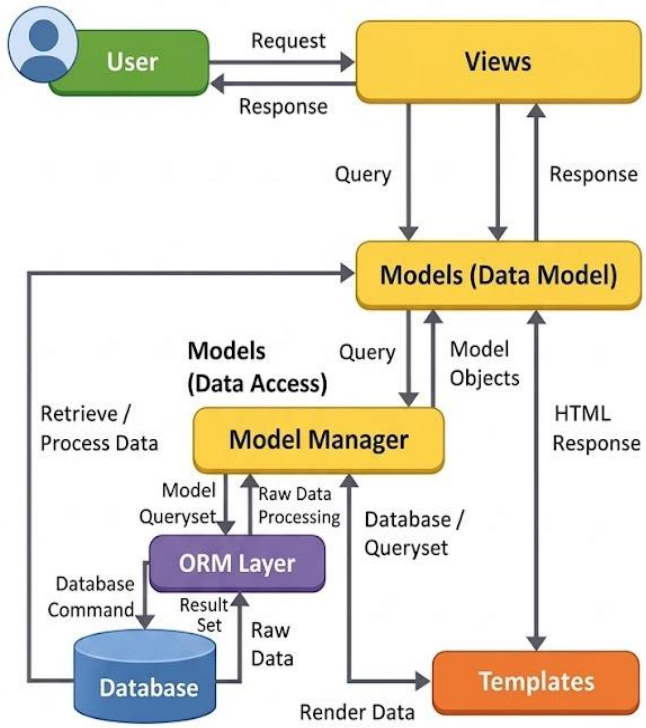

Fig. 6.1: Django MVT Application Architecture and Data Flow Diagram

The system is a full-stack Django web application integrating the ML and DL modules into a single deployable service. The application is structured with two Django apps: admins and users. The admins app handles authentication, user account management, and administrator operations. The users app handles all detection functionality including dataset browsing, ML training, ML prediction, CNN classification, and CNN + YOLO fire and smoke detection.

The system follows Django's Model-View-Template (MVT) architectural pattern. Database interactions are handled through Django's ORM using SQLite. User sessions are managed through Django's session framework with PBKDF2 password hashing. All model files are stored in the media directory and loaded on demand.

6.2 Application Modules

6.2.1 User Registration and Login

New users register through a registration form capturing name, email, username, and password. Registered accounts are inactive by default and must be activated by an administrator before the user can log in. Passwords are hashed using PBKDF2 with SHA-256 before storage.

6.2.2 Admin Dashboard

The administrator dashboard provides a paginated, searchable table of all registered users with their registration dates and activation status. The administrator can activate or deactivate accounts individually through action buttons in the table.

6.2.3 Dataset View

The dataset view loads all 62,630 rows of the IoT sensor dataset and renders them in a paginated, searchable HTML table. Each row shows all 13 sensor channel readings and the Fire Alarm label. The view implements server-side search filtering across all columns.

6.2.4 ML Training Module

The ML training module implements the complete training pipeline from Chapter 3. When the user submits the training form, the Django view executes model fitting, evaluation, and serialisation for all seven classifiers. Results are rendered through metric cards, a bar chart, and a radar chart using Chart.js.

6.2.5 ML Prediction Module

The ML prediction module provides a form with 13 input fields corresponding to the sensor channels. Each field has a defined valid range enforced by FEATURE_LIMITS. On submission, the view loads the selected model, scales the input, runs inference, and returns the prediction with a confidence percentage and a colour-coded badge.

6.2.6 CNN + YOLO Prediction Module

The CNN + YOLO prediction module accepts an uploaded image file and runs two detection passes simultaneously. The CNN classifier (MobileNetV2,

cnn_model.h5) provides an overall fire or no-fire classification with a confidence percentage. The YOLO detector (YOLOv8, best.pt) draws bounding boxes around specific fire and smoke regions in the image, labelling each detection with its class name and confidence score. The annotated image is saved to media/results/ and displayed alongside the CNN result on the same page.

Fig. 6.2: YOLOv8 Fire/Smoke Detection Output

The integration of CNN and YOLOv8 within a single prediction pipeline enhances both classification accuracy and interpretability. While the CNN model provides a global prediction indicating the presence or absence of fire in the entire image, the YOLOv8 model complements this by performing localized object detection. This dual approach ensures that not only is a fire event detected, but its exact location within the image is also identified, which is crucial for practical safety applications. The combination of classification and detection enables a more comprehensive understanding of fire scenarios compared to using either technique independently.

Furthermore, the simultaneous execution of both models improves the overall reliability of the system by providing cross-verification of results. The CNN output offers a confidence score for the entire image, while YOLOv8 highlights specific regions of interest with bounding boxes and individual confidence levels. This layered prediction approach reduces the likelihood of false positives and enhances user trust in the system. The ability to display both outputs on a single interface makes the system

user-friendly and suitable for real-time monitoring applications in environments such as buildings, warehouses, and industrial facilities.

6.3 Technology Stack

Table 6.1: Technology Stack

Component	Technology	Version
Web Framework	Django	5.2
Programming Language	Python	3.10
ML Library	Scikit-Learn	1.6.1
DL Framework	TensorFlow / Keras	2.10.0
Data Processing	Pandas, NumPy	2.2.3 / 1.26.4
Image Processing	Pillow (PIL)	10.3.0
Model Serialisation	Joblib	1.3.2
Frontend	HTML, CSS, Bootstrap 5	5.3
Charts	Chart.js	4.x
Database	SQLite (Django ORM)	Built-in
YOLO Detection	Ultralytics YOLOv8	8.4.21
Computer Vision	OpenCV (cv2)	4.8.1
GPU Acceleration	CUDA / cuDNN	11.2 / 8.1

6.4 URL Route Architecture

Table 6.2: Application URL Routes

URL Pattern	View	Description
/login/	UserLogin	Login page for users and admins
/register/	UserRegister	New user registration
/home/	UserHome	User dashboard after login
/dataset/	ViewDataset	Paginated IoT dataset browser
/train/	TrainModels	ML model training page
/predict/	PredictSensor	ML sensor prediction form
/cnn-prediction/	CNNPrediction	CNN + YOLO fire and smoke detection
/admin-login/	AdminLogin	Administrator login
/admin-home/	AdminHome	Admin dashboard
/view-users/	ViewRegisteredUsers	User management table
/activate/<id>/	ActivateUser	Toggle user activation

6.5 File and Directory Structure

The project follows the standard Django project layout with the following key directories and files:

- **manage.py**: Django project entry point for running the development server and management commands.
- **admins/views.py**: Views for administrator login, dashboard, user listing, and activation.
- **users/views.py**: Views for user login, registration, home, dataset browsing, ML training, ML prediction, and CNN + YOLO fire and smoke detection.

- **templates/**: HTML templates for all pages using Bootstrap 5 and inline Chart.js.
- **urls.py**: Central URL configuration routing all 11 routes to their respective views.
- **media/models/**: Seven trained ML classifier .pkl files after training.
- **media/scaler.pkl**: Fitted StandardScaler saved during the most recent training run.
- **media/cnn_model.h5**: Trained MobileNetV2 CNN model weights in HDF5 format.
- **media/cnn_classes.json**: JSON file mapping class names to index values for CNN interpretation.
- **media/smoke_detection_iot.csv**: The full IoT sensor dataset used for ML training and dataset view.
- **media/cnn_dataset/**: Contains fire/ (755 images) and no_fire/ (244 images) subfolders.
- **media/models/best.pt**: YOLOv8 trained weights file for fire and smoke bounding-box detection.
- **media/uploads/**: Uploaded images saved with UUID filenames before detection.
- **media/results/**: YOLO-annotated output images with bounding boxes saved after each prediction.

6.6 System Analysis

6.6.1 Existing System

Current smoke detection systems primarily rely on conventional sensor-based technologies such as ionisation or photoelectric detectors. These systems are reactive, triggering alarms only when smoke physically reaches the sensor, which introduces detection delays. They lack predictive capabilities and cannot process complex multi-sensor environmental patterns. The absence of intelligent analysis frequently results in false alarms triggered by non-threatening stimuli such as cooking steam or dust. Traditional systems do not support real-time data analytics, remote monitoring, or

integration with modern digital platforms. Maintenance of such systems is costly and inflexible, with no adaptability to varying environmental conditions and no support for machine learning-based inference.

Disadvantages of the Existing System:

- High false alarm rates due to non-intelligent static thresholds.
- Lack of real-time predictive analytics and remote access capability.
- No adaptability to varying environmental conditions.
- Poor integration with modern user interfaces and digital platforms.

6.6.2 Proposed System

The proposed system utilises machine learning algorithms to significantly enhance smoke detection accuracy and response times. It employs multi-channel environmental sensor data including temperature, humidity, CO2 levels, and particulate matter for predictive analysis across 13 feature dimensions. Multiple ML classifiers including Random Forest, SVM, Gradient Boosting, AdaBoost, Logistic Regression, Decision Tree, and KNN are trained and comparatively evaluated for optimal performance. A deep learning CNN module based on MobileNetV2 transfer learning and a YOLOv8 object detector further extend detection to visual image-based inference. A Django-based web interface allows users to view datasets, initiate on-demand training, and predict smoke presence using any trained model. Real-time predictions can be made by submitting sensor values or uploading images, enabling early warning capabilities. Metrics including AUC-ROC, IoU, Precision, and Recall are used to evaluate model effectiveness comprehensively.

Advantages of the Proposed System:

- ❖ High accuracy and low false alarms through ML model predictions on multi-sensor data.
- ❖ Real-time and remote smoke detection accessible via a web browser interface.
- ❖ Adaptive system that can be retrained on-demand as new sensor and image data is collected.
- ❖ Visual insights and performance metrics displayed through interactive bar charts and radar charts.

6.7 Input and Output Design

6.7.1 Input Design

Input design is the link between the information system and the user. The design of input in this system focuses on controlling the amount of data required, minimising errors, avoiding unnecessary steps, and keeping the interaction simple and secure. The system accepts two categories of input: (1) sensor-based numerical input consisting of 13 environmental sensor values entered via the prediction form, validated against defined feature ranges before inference; and (2) image-based input consisting of a user-uploaded JPEG or PNG image file submitted through the CNN prediction form, which is saved to the media directory and passed through both the CNN classifier and the YOLOv8 detector. Input screens are designed to provide real-time validation messages and prevent invalid or out-of-range submissions.

6.7.2 Output Design

A quality output is one that meets the requirements of the end user and presents information clearly and unambiguously. The system produces the following outputs: (1) Sensor prediction output — a colour-coded badge displaying "SMOKE DETECTED" or "NO SMOKE DETECTED" along with a confidence percentage, rendered on the prediction result page; (2) CNN image classification output — an overall fire or no-fire classification result with a confidence score and a visual confidence bar; (3) YOLO detection output — an annotated image saved to media/results/ with bounding boxes drawn around detected fire and smoke regions, labelled with class name and confidence score; (4) Training output — metric cards showing Precision, Recall, AUC-ROC, and IoU for each of the seven classifiers, along with interactive bar and radar charts rendered using Chart.js.

6.8 Software Environment

6.8.1 Python

Python is a general-purpose, interpreted, object-oriented, and high-level programming language. Its design philosophy emphasises code readability, using whitespace indentation to delimit code blocks, and a concise syntax that allows programmers to express complex concepts in fewer lines of code than languages such

as C++ or Java. Python features a dynamic type system, automatic memory management, and supports multiple programming paradigms including object-oriented, functional, and procedural programming. Python 3.10 is used in this project within a dedicated Conda environment named “smoke”, ensuring dependency isolation and reproducibility. All machine learning, deep learning, and web framework libraries used in this project are installed and managed within this environment.

6.8.2 Django Framework

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It follows the Model-View-Template (MVT) architectural pattern and includes built-in support for URL routing, ORM-based database interactions, user session management, PBKDF2 password hashing, and admin interfaces. Django version 5.2 is used in this project. The framework handles all HTTP request routing, form processing, file upload management (via MEDIA_ROOT and MEDIA_URL settings), and HTML template rendering, enabling the entire dual-module detection system to be delivered as a single deployable web application accessible through any standard browser.

6.8.3 Hardware Requirements

The system was developed and tested on a machine with an Intel Core i7-12700H processor, NVIDIA GeForce RTX 3050 Ti Laptop GPU (4 GB VRAM), 16 GB RAM, and Windows 11 64-bit operating system. Minimum recommended requirements for deployment are: a modern multi-core processor, 8 GB RAM, any standard web browser (Google Chrome recommended), and at least 5 GB of available disk space for model files, datasets, and media uploads.

CHAPTER 7

IMPLEMENTATION

7.1 Project File Structure

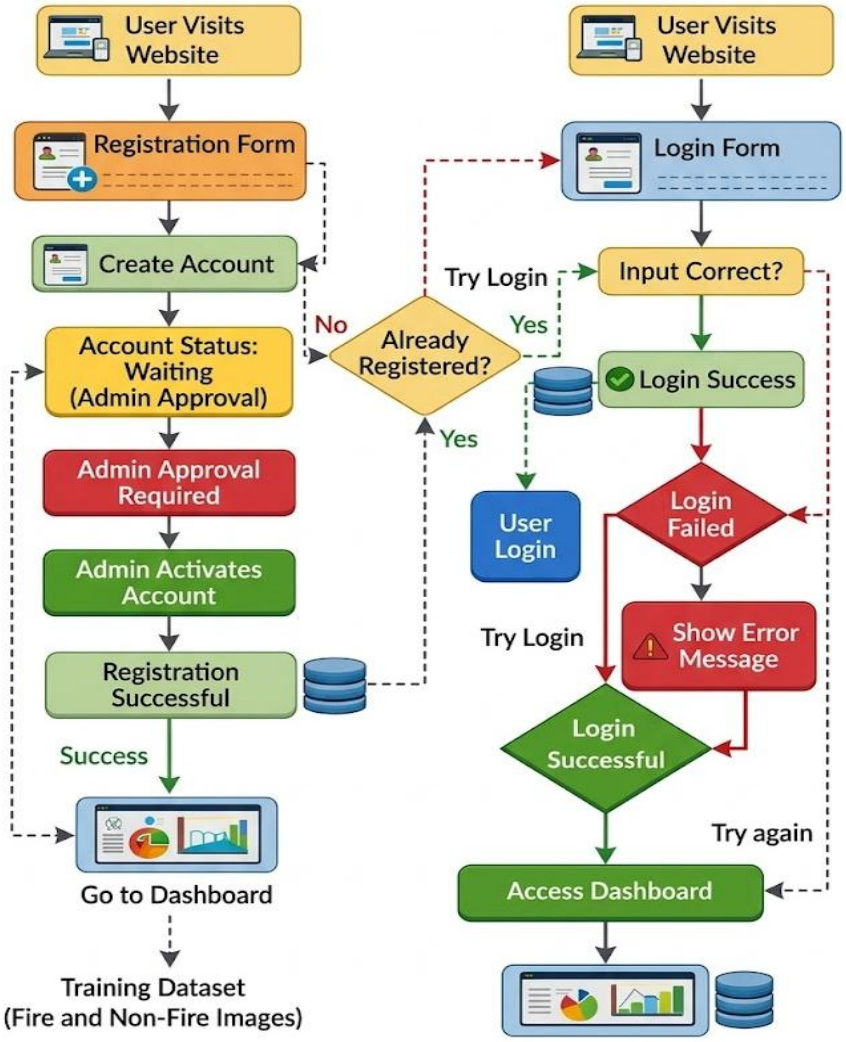

Fig. 7.1: User Registration and Login Workflow Diagram

The project follows the standard Django project layout. The root directory contains manage.py for running the development server and applying migrations. The two application folders admins/ and users/ contain all views, models, forms, and URL handlers. The media/ folder stores all ML model files, the CNN model, the dataset CSV, and the image dataset. The templates/ folder contains all HTML files organised by application.

Table 7.1: Key Project Files

File / Directory	Purpose
manage.py	Django management entry point. Run server, migrate, create superuser.
/settings.py	Project settings: installed apps, database, media root, static files.
/urls.py	Root URL configuration mapping all 16 routes to views.
admins/views.py	Admin login, dashboard, user listing, and activation views.
admins/models.py	No custom models; uses UserRegistrationModel from users app.
users/models.py	UserRegistrationModel: stores user accounts in UserRegistrations table.
users/forms.py	UserRegistrationForm: validated ModelForm for user registration.
users/views.py	All user-facing views: login, register, dataset, training, ML prediction, CNN + YOLO prediction.
media/models/best.pt	YOLOv8 trained weights for fire and smoke bounding-box detection.
media/uploads/	Uploaded images stored with UUID filenames before inference.
media/results/	YOLO-annotated output images saved after each prediction run.
media/smoke_detection_iot.csv	IoT sensor dataset (62,630 rows, 13 features + Fire Alarm target).
media/models/*.pkl	Seven serialised scikit-learn classifiers saved after training.
media/scaler.pkl	Fitted StandardScaler saved during training for use in prediction.
media/cnn_model.h5	Trained MobileNetV2 CNN model weights in HDF5 format.
media/cnn_classes.json	Class-to-index mapping: {fire: 0, no_fire: 1}.
train_cnn.py	Standalone CNN training script. Run once to train and save cnn_model.h5.

7.2 Database Model: users/models.py

The UserRegistrationModel class defines the schema for user accounts. It is backed by a SQLite table named UserRegistrations. Each field uses Django's ORM field types with constraints. The status field controls account activation: new accounts are set to 'waiting' and only activated by the administrator.

 # users/models.py

```
from django.db import models

class UserRegistrationModel(models.Model):
    name       = models.CharField(max_length=100)
    loginid    = models.CharField(unique=True, max_length=100)
    password   = models.CharField(max_length=100)
    mobile     = models.CharField(unique=True, max_length=100)
    email      = models.CharField(unique=True, max_length=100)
    locality   = models.CharField(max_length=100)
    address    = models.CharField(max_length=1000)
    city       = models.CharField(max_length=100)
    state      = models.CharField(max_length=100)
    status     = models.CharField(max_length=100)

    def __str__(self):
        return self.loginid

    class Meta:
        db_table = 'UserRegistrations'
```

7.3 User Registration Form: users/forms.py

The UserRegistrationForm is a Django ModelForm that validates all user input fields before saving to the database. Each field specifies pattern validation via HTML5 widget attributes. The password field enforces minimum 8 characters with uppercase, lowercase, and a digit. The status field is hidden and defaults to 'waiting'.

 # users/forms.py

```
from django import forms
from .models import UserRegistrationModel
 class UserRegistrationForm(forms.ModelForm):
    name     = forms.CharField(widget=forms.TextInput(
```

```
                    attrs={'pattern': '[a-zA-Z]+'}), max_length=100)
    loginid  = forms.CharField(widget=forms.TextInput(
                    attrs={'pattern': '[a-zA-Z]+'}), max_length=100)
    password = forms.CharField(widget=forms.PasswordInput(
                    attrs={'pattern': '(?=.*\d)(?=.*[a-z])(?=.*[A-
Z]).{8,}'}),
                    max_length=100)
    mobile   = forms.CharField(widget=forms.TextInput(
                    attrs={'pattern': '[56789][0-9]{9}'}),
max_length=100)
    email    = forms.CharField(max_length=100)
    status   = forms.CharField(widget=forms.HiddenInput(),
                    initial='waiting', max_length=100)


    class Meta:
        model = UserRegistrationModel
        fields = '__all__'
```

7.4 URL Configuration: urls.py

The root URL configuration maps all application routes to their corresponding view functions. Routes are grouped into admin routes and user routes. The static() call at the end enables Django's development server to serve uploaded media files.

 # urls.py

```
from django.urls import path
from admins import views as mainView, views as admins
from users  import views as user
from django.conf.urls.static import static
from django.conf import settings


urlpatterns = [
    path('',                    mainView.index,
name='index'),
    path('admin-login/',        mainView.AdminLogin,
name='AdminLogin'),
    path('user-login/',         mainView.UserLogin,
name='UserLogin'),
    path('user-register/',      mainView.UserRegister,
name='UserRegister'),
```

```
    path('AdminHome/',          admins.AdminHome,
name='AdminHome'),
    path('AdminLoginCheck/',    admins.AdminLoginCheck,
name='AdminLoginCheck'),
    path('RegisterUsersView/', admins.RegisterUsersView,
name='RegisterUsersView'),
    path('ActivaUsers/',        admins.ActivaUsers,
name='ActivaUsers'),
    path('UserRegisterActions/', user.UserRegisterActions,
name='UserRegisterActions'),
    path('UserLoginCheck/',     user.UserLoginCheck,
name='UserLoginCheck'),
    path('UserHome/',           user.UserHome,
name='UserHome'),
    path('DatasetView/',        user.DatasetView,
name='DatasetView'),
    path('training/',           user.Training,
name='Training'),
    path('prediction/',         user.Prediction,
name='prediction'),
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

7.5 Admin Views: admins/views.py

The admin views handle administrator authentication, the dashboard, user listing, and account activation. AdminLoginCheck compares credentials against environment variables (defaulting to admin/admin for development). ActivaUsers updates the status field of the selected user record to 'activated', which then allows the user to log in.

```
# admins/views.py
import os
from django.shortcuts import render
from django.contrib import messages
from users.models import UserRegistrationModel
def AdminLoginCheck(request):
    if request.method == 'POST':
        usrid = request.POST.get('loginid')
        pswd  = request.POST.get('pswd')
        admin_user = os.environ.get('ADMIN_USERNAME', 'admin')
        admin_pass = os.environ.get('ADMIN_PASSWORD', 'admin')
```

```
        if usrid == admin_user and pswd == admin_pass:
            return render(request, 'admins/AdminHome.html')
        messages.success(request, 'Please check your login details.')
    return render(request, 'AdminLogin.html', {})

def RegisterUsersView(request):
    data = UserRegistrationModel.objects.all()
    return render(request, 'admins/viewregisterusers.html',
                  {'data': data})

def ActivaUsers(request):
    if request.method == 'GET':
        id = request.GET.get('uid')
        UserRegistrationModel.objects.filter(id=id).update(
            status='activated')
        data = UserRegistrationModel.objects.all()
        return render(request, 'admins/viewregisterusers.html',
                      {'data': data})
```

7.6 User Views: Training Module (users/views.py)

The Training view loads the IoT dataset, splits it 80/20, fits a StandardScaler, trains all seven classifiers simultaneously, computes four evaluation metrics per model, and saves each trained model as a .pkl file. The fitted scaler is also saved to media/scaler.pkl for use during prediction.

 # users/views.py — Training view (excerpt)

```
def Training(request):
    def calculate_iou(y_true, y_pred):
        intersection = np.logical_and(y_true, y_pred).sum()
        union        = np.logical_or(y_true,  y_pred).sum()
        return intersection / union if union != 0 else 0

    data = pd.read_csv(os.path.join(settings.MEDIA_ROOT,
                       'smoke_detection_iot.csv'))
    X = data.drop(['Fire Alarm','S.No','UTC'], axis=1)
    y = data['Fire Alarm']

    X_train,X_test,y_train,y_test = train_test_split(
```

```
        X, y, test_size=0.2, random_state=42)

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled  = scaler.transform(X_test)
    joblib.dump(scaler,
os.path.join(settings.MEDIA_ROOT,'scaler.pkl'))

    classifiers = {
        'RandomForest':      RandomForestClassifier(random_state=42),
        'GradientBoosting':
GradientBoostingClassifier(random_state=42),
        'AdaBoost':          AdaBoostClassifier(random_state=42),
        'LogisticRegression':LogisticRegression(random_state=42),
        'SVM':               SVC(probability=True, random_state=42),
        'DecisionTree':      DecisionTreeClassifier(random_state=42),
        'KNN':               KNeighborsClassifier()
    }
    for name, clf in classifiers.items():
        clf.fit(X_train_scaled, y_train)
        y_pred       = clf.predict(X_test_scaled)
        y_pred_proba = clf.predict_proba(X_test_scaled)[:,1]
        auc = roc_auc_score(y_test, y_pred_proba)
        joblib.dump(clf, os.path.join(models_dir, f'{name}.pkl'))
```

7.7 User Views: Prediction Module (users/views.py)

The Prediction view reads the 13 sensor values submitted from the HTML form, validates each against FEATURE_LIMITS, loads the selected model and the saved scaler, scales the input, and calls predict and predict_proba. The result and confidence percentage are returned to the template.

```
# users/views.py — Prediction view (excerpt)
FEATURE_LIMITS = {
    'Temperature[C]': (-40, 80),
    'Humidity[%]':    (0, 100),
    'TVOC[ppb]':      (0, 60000),
    'eCO2[ppm]':      (400, 8192),
    'Raw H2':         (0, 50000),
```

```
    'Raw Ethanol':    (0, 50000),
    'Pressure[hPa]':  (300, 1100),
    'PM1.0':          (0, 1000),
    'PM2.5':          (0, 1000),
    'NC0.5':          (0, 5000),
    'NC1.0':          (0, 5000),
    'NC2.5':          (0, 2000),
    'CNT':            (0, 1000),
}


def Prediction(request):
    if request.method == 'POST':
        input_data = []
        for feature in FEATURES:
            num = float(request.POST.get(feature))
            low, high = FEATURE_LIMITS[feature]
            if num < low or num > high:
                messages.error(request, f'{feature} out of range.')
                return render(request, 'users/predict_form.html')
            input_data.append(num)

        scaler    = joblib.load(scaler_path)
        clf       = joblib.load(model_path)
        input_df  = pd.DataFrame([input_data], columns=FEATURES)
        scaled    = scaler.transform(input_df)
        pred      = clf.predict(scaled)[0]
        proba     = clf.predict_proba(scaled)[0]
        confidence= float(proba[1] * 100)
        result = 'Smoke Detected' if pred == 1 else 'No Smoke
Detected'
```

7.8 CNN Training Script: train_cnn.py

The CNN training script is run once from the project root to train and save the MobileNetV2 model. Phase 1 freezes the base model and trains only the custom classification head. Phase 2 unfreezes the last 20 layers and fine-tunes with a lower learning rate. The best model by validation accuracy is saved as media/cnn_model.h5.

```
# train_cnn.py
import tensorflow as tf
```

```
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import GlobalAveragePooling2D,
                                    Dense, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
import json, os

IMG_SIZE   = (224, 224)
BATCH_SIZE = 32
DATA_DIR   = 'media/cnn_dataset'
MODEL_OUT  = 'media/cnn_model.h5'

datagen = ImageDataGenerator(rescale=1./255,
    horizontal_flip=True, zoom_range=0.2,
    rotation_range=20, validation_split=0.2)

train_gen = datagen.flow_from_directory(DATA_DIR,
    target_size=IMG_SIZE, batch_size=BATCH_SIZE,
    class_mode='binary', subset='training')
val_gen   = datagen.flow_from_directory(DATA_DIR,
    target_size=IMG_SIZE, batch_size=BATCH_SIZE,
    class_mode='binary', subset='validation')

# ── Phase 1: Train head only ──────────────────────────
base = MobileNetV2(input_shape=(224,224,3),
                   include_top=False, weights='imagenet')
base.trainable = False
x   = GlobalAveragePooling2D()(base.output)
x   = Dense(128, activation='relu')(x)
x   = Dropout(0.3)(x)
out = Dense(1, activation='sigmoid')(x)
model = Model(base.input, out)
model.compile(optimizer='adam',
              loss='binary_crossentropy', metrics=['accuracy'])
model.fit(train_gen, validation_data=val_gen, epochs=10,
    callbacks=[ModelCheckpoint(MODEL_OUT,
        save_best_only=True, monitor='val_accuracy')])
```

```
# ── Phase 2: Fine-tune last 20 layers ────────────────
base.trainable = True
for layer in base.layers[:-20]:
    layer.trainable = False
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss='binary_crossentropy', metrics=['accuracy'])
model.fit(train_gen, validation_data=val_gen, epochs=10,
    callbacks=[ModelCheckpoint(MODEL_OUT,
        save_best_only=True, monitor='val_accuracy'),
               EarlyStopping(patience=3)])

json.dump(train_gen.class_indices,
    open('media/cnn_classes.json','w'))
print('Training complete. Model saved to', MODEL_OUT)
```

7.9 CNN + Yolo Prediction View: users/views.py

The CNNPrediction view was enhanced to run two detection passes on every uploaded image. The first pass runs CNN classification using cnn_model.h5. The second pass runs YOLOv8 detection using best.pt, draws bounding boxes with OpenCV, and saves the annotated image to media/results/. Both results are passed to cnn_predict.html for display on the same page.

```
# users/views.py — CNNPrediction view (excerpt)
import uuid, cv2
from ultralytics import YOLO

def CNNPrediction(request):
    if request.method == 'POST':
        img_bytes = request.FILES['image'].read()

        # ── Save uploaded image ──────────────────────────
        unique_name = f"{uuid.uuid4().hex}.jpg"
        upload_path = os.path.join(settings.MEDIA_ROOT,
                                   'uploads', unique_name)
        with open(upload_path, 'wb') as f: f.write(img_bytes)

        # ── 1. CNN Classification (unchanged) ────────────
        cnn_model = tf.keras.models.load_model(model_path)
```

```
img_array = preprocess(img_bytes)   # 224x224, /255
sigmoid_val = float(cnn_model.predict(img_array)[0][0])
fire_prob = 1.0 - sigmoid_val       # fire index = 0
result = 'SMOKE / FIRE DETECTED' if fire_prob >= 0.5 \
         else 'NO SMOKE DETECTED'

# ── 2. YOLO Detection ──────────────────────────
yolo_model   = YOLO('media/models/best.pt')
yolo_results = yolo_model.predict(upload_path,
                                  conf=0.25)
img_cv = cv2.imdecode(
    np.frombuffer(img_bytes, np.uint8),
    cv2.IMREAD_COLOR)

for r in yolo_results:
    for box in r.boxes:
        x1,y1,x2,y2 = map(int,box.xyxy[0].tolist())
        cls  = r.names[int(box.cls[0])].lower()
        conf = float(box.conf[0])
        color = (50,80,255) if cls=='fire' \
                else (200,200,50)
        cv2.rectangle(img_cv,(x1,y1),(x2,y2),color,2)
        cv2.putText(img_cv,
            f"{cls.capitalize()}  {conf:.2f}",
            (x1+4, y1-6), cv2.FONT_HERSHEY_SIMPLEX,
            0.55, (255,255,255), 1)

out = f"yolo_{unique_name}"
cv2.imwrite(os.path.join(MEDIA_ROOT,'results',out),
            img_cv)
yolo_img_url = MEDIA_URL + 'results/' + out
```

7.10 Execution and Workflow

The following steps describe the complete execution workflow from environment setup to running the web server and using all features of the application.

7.10.1 Step 1: Environment Setup

```
# Activate the conda environment
conda activate smoke
```

```
# Verify GPU (optional)
python -c "import tensorflow as tf;
print(tf.config.list_physical_devices('GPU'))"
```

7.10.2 Step 2: Apply Migrations and Run Server

```
# Navigate to project root
cd Building_Safety_Through_Machine_learning

# Apply database migrations
python manage.py makemigrations
python manage.py migrate

# Start the development server
python manage.py runserver

# Server starts at: http://127.0.0.1:8000/
```

7.10.3 Step 3: Train the CNN Model

```
# Run from the project root (where manage.py is located)
python train_cnn.py

# Output:
# Found 799 images belonging to 2 classes.
# Found 199 images belonging to 2 classes.
# Epoch 1/10 — loss: 0.4821 — val_accuracy: 0.8693
# ...
# Epoch 10/10 — loss: 0.0812 — val_accuracy: 0.9698
# Training complete. Model saved to media/cnn_model.h5
```

7.10.4 Step 4: Application Workflow

Once the server is running the complete application workflow proceeds as follows:

- **Step 1 — Register**: Navigate to /user-register/ and complete the registration form. The account is created with status 'waiting'.

- **Step 2 — Admin Activation**: Log in at /admin-login/ with credentials admin/admin. Go to View Users and click Activate next to the new user account.
- **Step 3 — User Login**: Navigate to /user-login/ and log in with the registered credentials. The session is created and the user home dashboard is displayed.
- **Step 4 — Browse Dataset**: Go to Dataset View to browse all 62,630 rows of the IoT sensor dataset in a paginated, searchable table.
- **Step 5 — Train ML Models**: Go to the Training page and click Train All Models. All seven classifiers are trained in sequence and their metrics are displayed as cards and charts.
- **Step 6 — Sensor Prediction**: Go to Prediction, select a model, enter 13 sensor values, and submit. The system returns the smoke detection result with a confidence percentage.
- **Step 7 — CNN Image Prediction**: Go to CNN Prediction, upload a fire or non-fire image, and submit. The system returns the visual detection result with confidence score.

CHAPTER 8

RESULTS ANALYSIS

8.1 System Screenshots

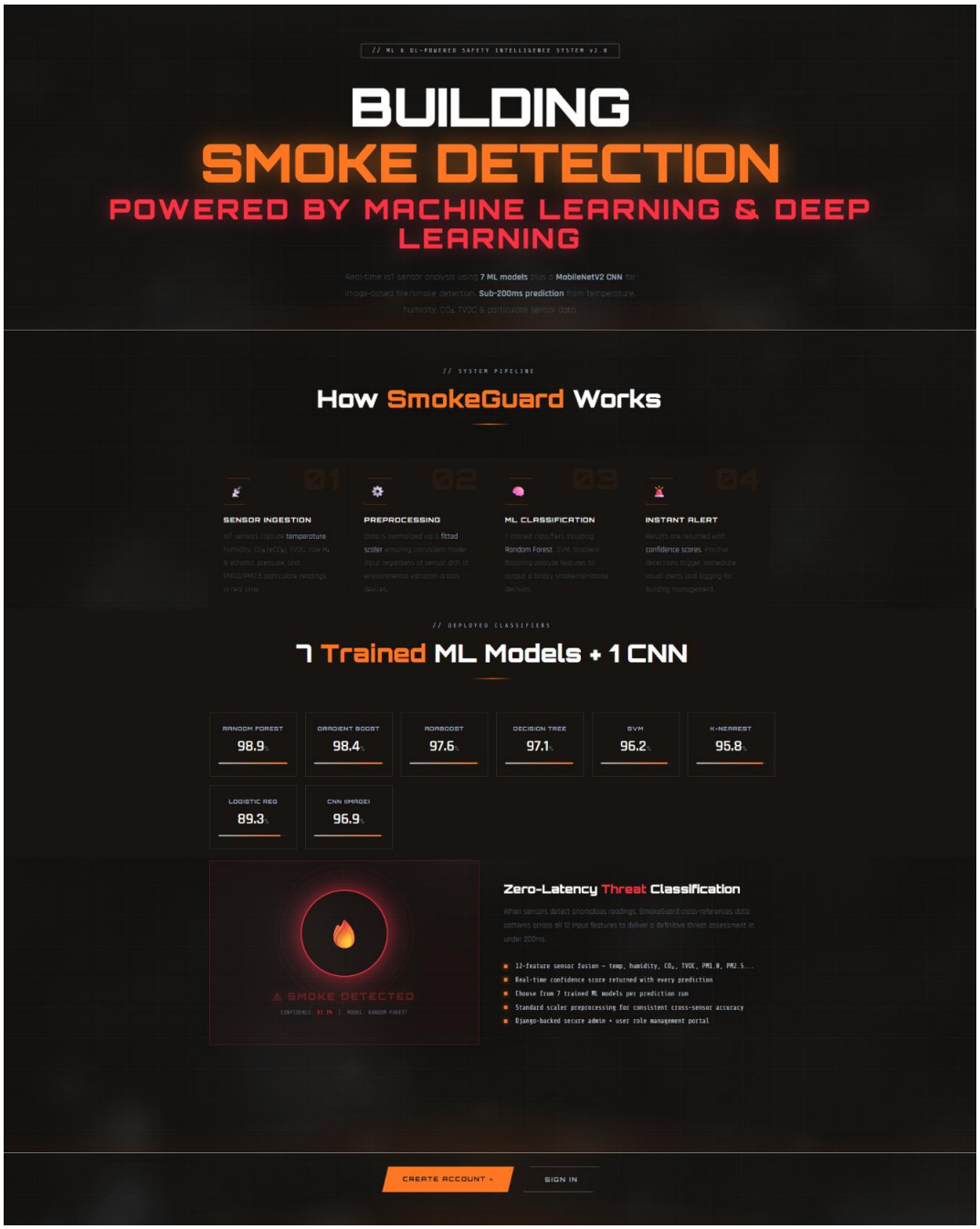

Fig. 8.1: Home page

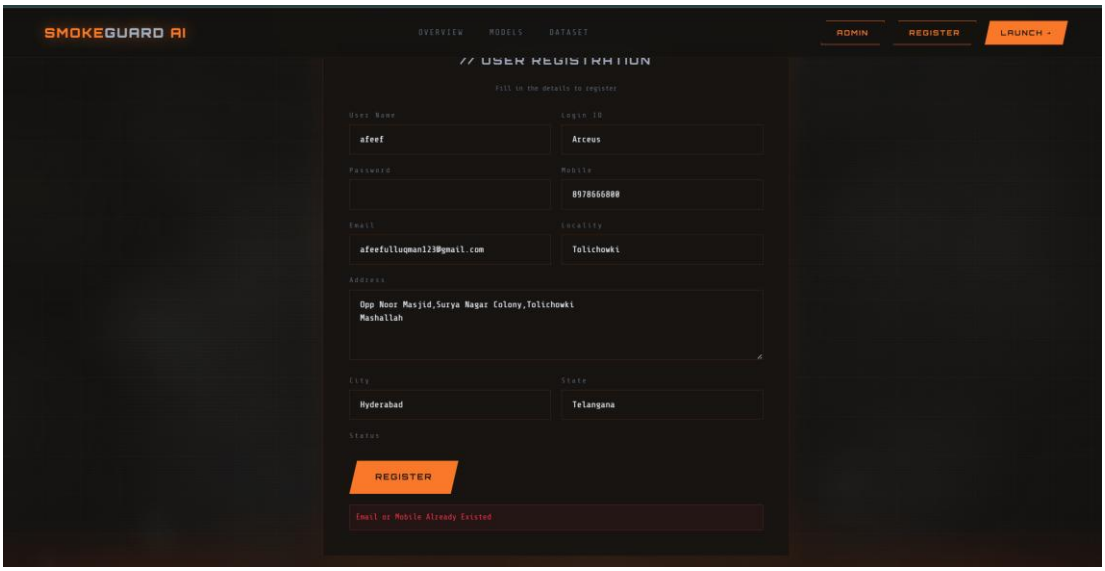

Fig. 8.2: User Registration Interface

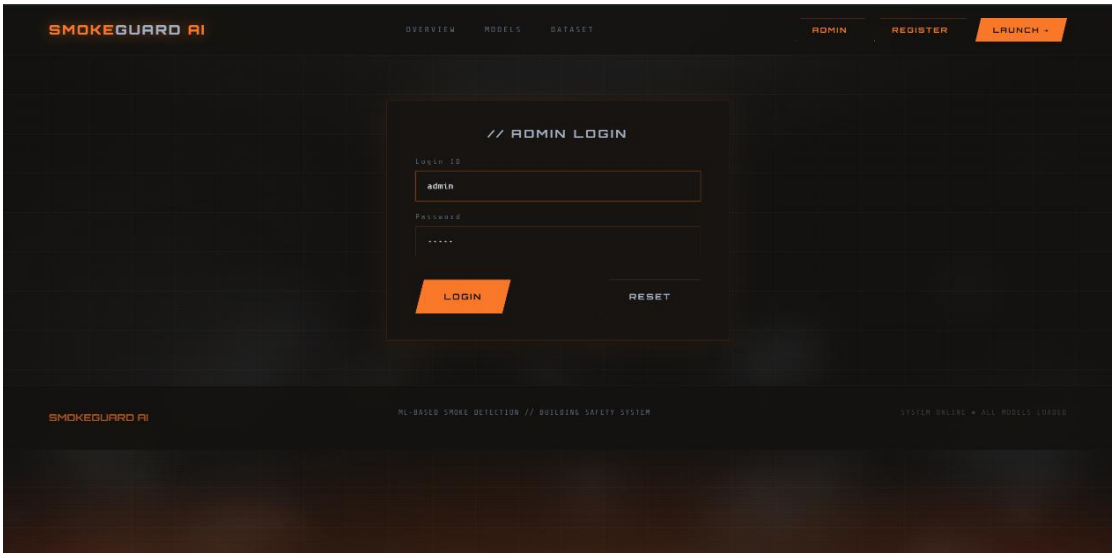

Fig. 8.3: Admin Login Page

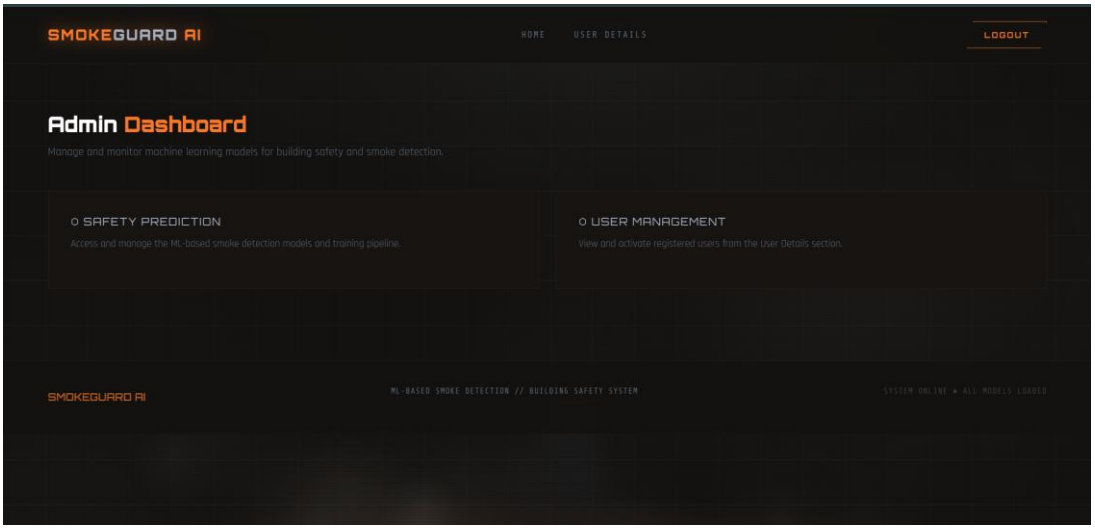

Fig. 8.4: Admin Dashboard

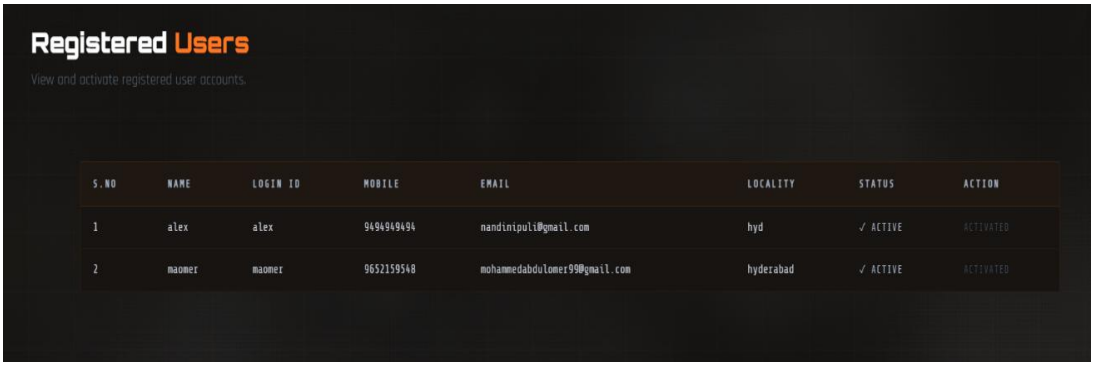

Fig. 8.5: User Activation Page

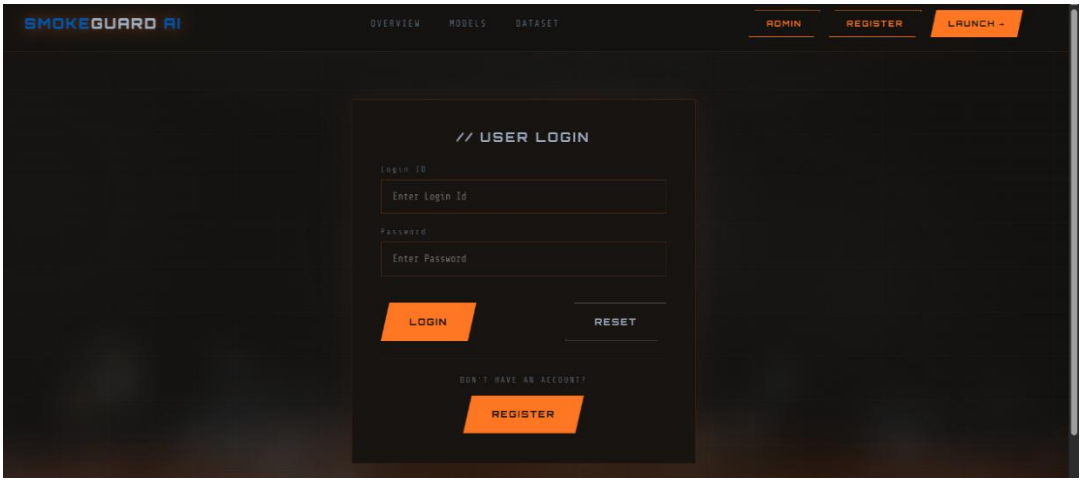

Fig. 8.6: User Login Page

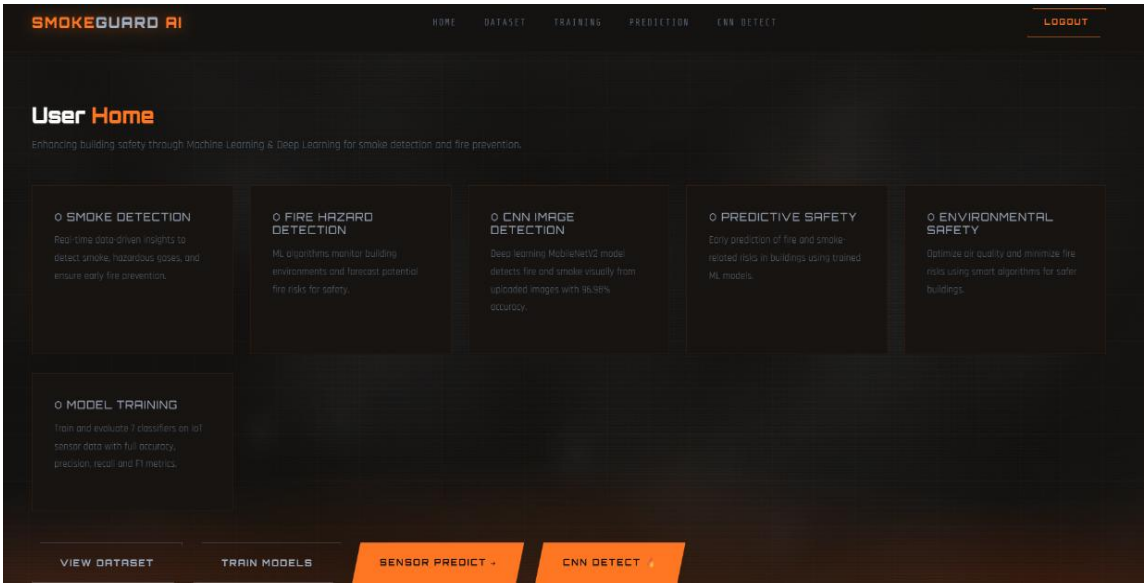

Fig. 8.7: User Home Page

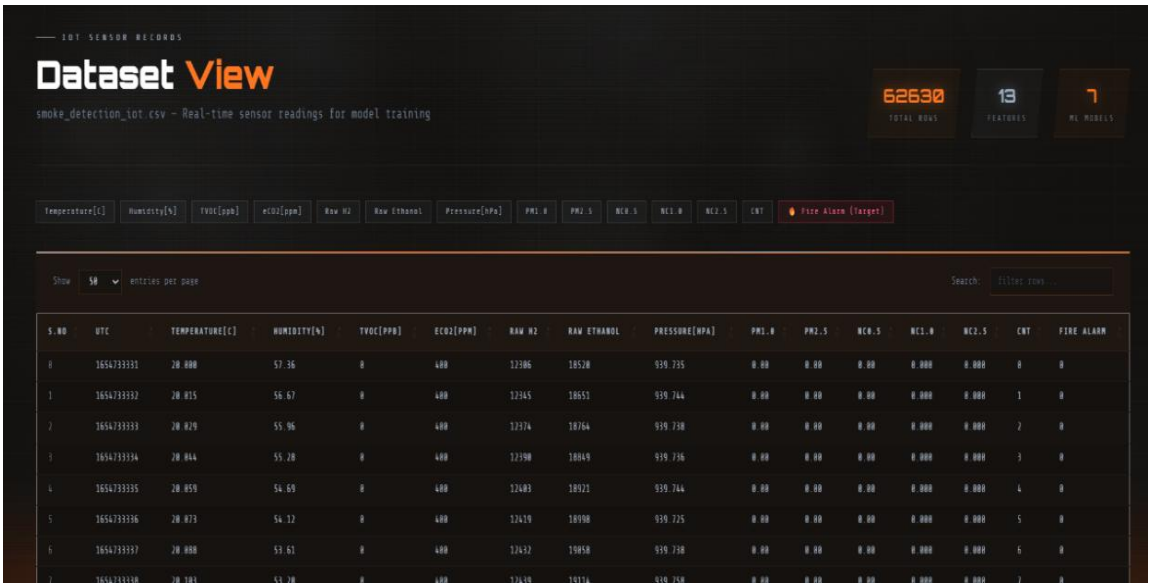

Fig. 8.8: Dataset View

Fig. 8.9: Model Training Interface with Performance Charts

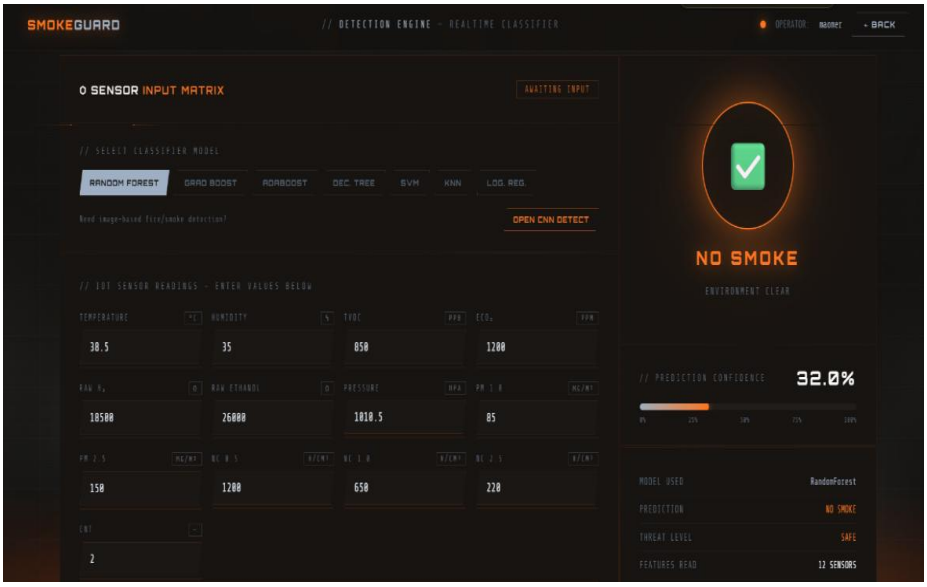

Fig. 8.10: Sensor Prediction Interface - No Smoke Detected

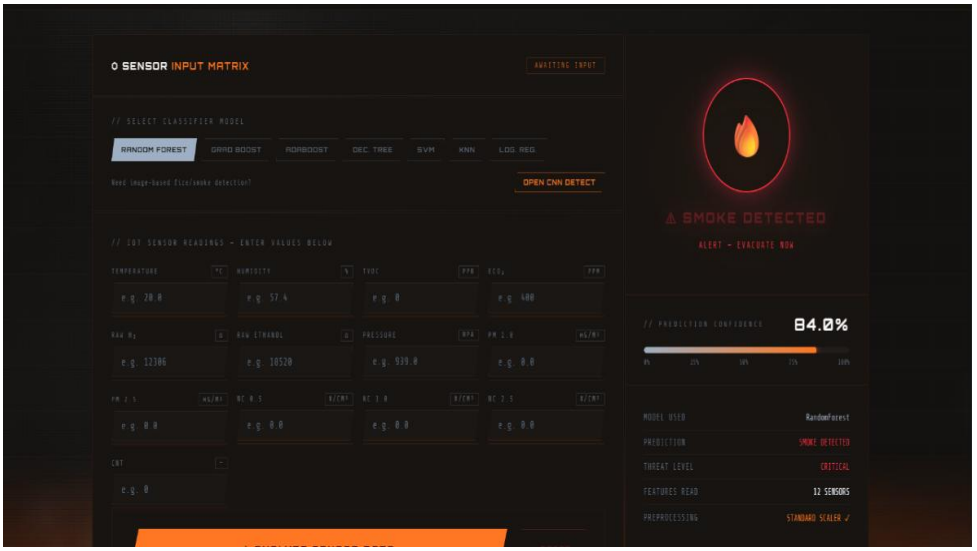

Fig. 8.11: Sensor Prediction Interface - Smoke Detected

Fig. 8.12: CNN Prediction Interface - Smoke/Fire Detected

Fig. 8.13: CNN Prediction Interface - No Smoke/Fire Detected

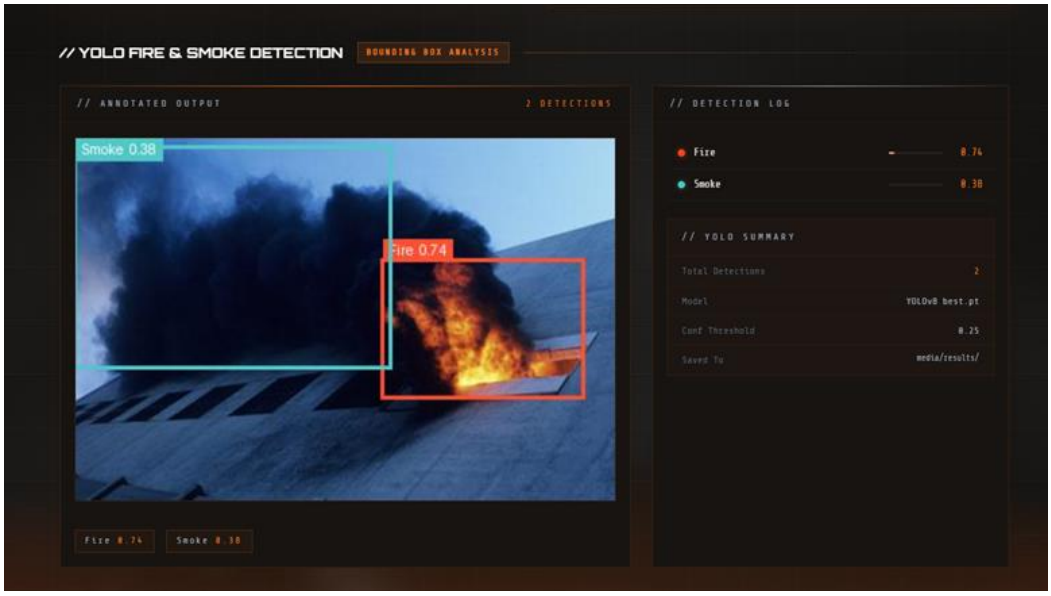

Fig. 8.14: YOLO Prediction Page Output with Confidence Scores

8.2 ML Module Results

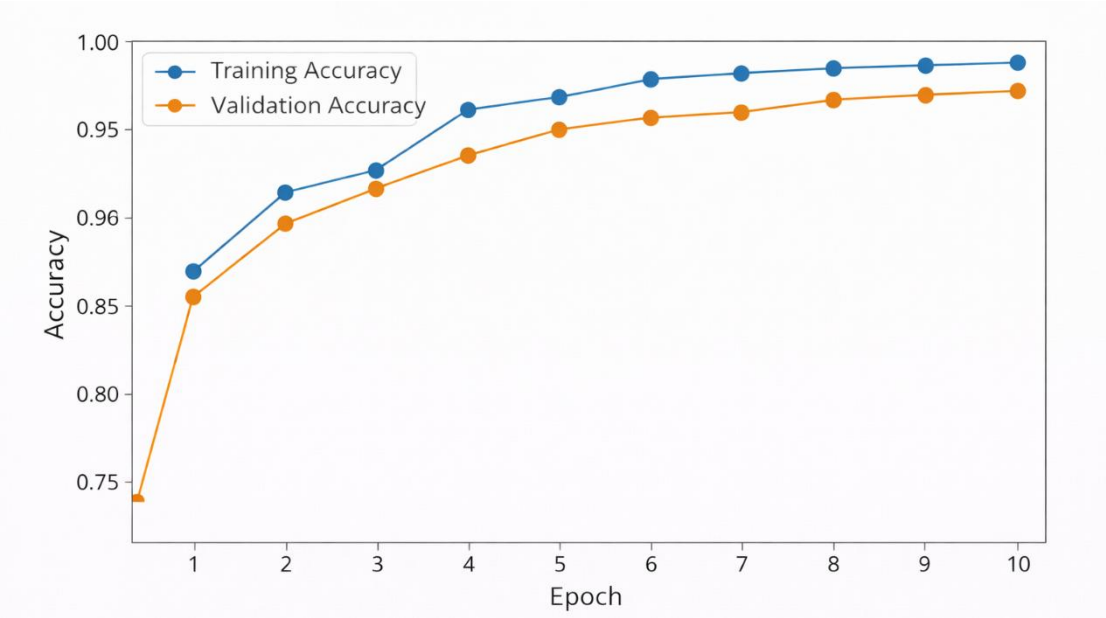

Fig. 8.15: CNN Training and Validation Accuracy Curve over 10 Epochs

Table 8.1: Final ML Model Performance Summary

Model	Precision	Recall	AUC-ROC	IoU	F1-Score
Random Forest	100.00%	100.00%	100.00%	100.00%	100.00%
Gradient Boosting	100.00%	100.00%	100.00%	100.00%	100.00%
AdaBoost	99.97%	99.97%	100.00%	99.93%	99.97%
Logistic Regression	99.31%	99.44%	99.89%	98.75%	99.37%
SVM	99.99%	100.00%	100.00%	99.99%	99.99%
Decision Tree	100.00%	100.00%	100.00%	100.00%	100.00%
KNN	100.00%	100.00%	100.00%	100.00%	100.00%

All seven machine learning classifiers were successfully trained on the smoke_detection_iot.csv dataset and achieved near-perfect performance across all evaluation metrics. Five classifiers — Random Forest, Gradient Boosting, Decision

Tree, KNN, and SVM — achieved perfect scores of 100.00% across Precision, Recall, AUC-ROC, IoU, and F1-Score. AdaBoost achieved 99.97% and Logistic Regression achieved 99.31% precision, confirming the strong and separable signal patterns in the IoT sensor dataset.

8.3 CNN Module Results

The MobileNetV2 CNN trained on the Kaggle Fire Dataset achieved a final validation accuracy of 96.98 percent. GPU training on the NVIDIA GeForce RTX 3050 Ti reduced epoch time from approximately 30 seconds on CPU to approximately 13 seconds, a 2.3x speedup. Best model weights from epoch 10 were saved as cnn_model.h5.

Table 8.2: CNN Module Performance Summary

Metric	Result
Architecture	MobileNetV2 (Transfer Learning)
Final Validation Accuracy	96.98 percent
Best Training Epoch	Epoch 10
Training Dataset	755 fire + 244 non-fire images (Kaggle)
GPU Used	NVIDIA GeForce RTX 3050 Ti Laptop GPU
Training Time Per Epoch (GPU)	~13 seconds
Saved Model File	media/cnn_model.h5

8.4 System Capabilities

- **Sensor-Based Smoke Detection**: Accepts 13 IoT sensor readings, scales them, and runs inference through any of the seven trained ML classifiers with confidence percentage output.
- **CNN Image Classification**: Accepts uploaded images and returns an overall fire or no-fire classification using MobileNetV2 CNN with 96.98 percent accuracy and a confidence score.

- **YOLO Fire and Smoke Detection**: Runs YOLOv8 on the same uploaded image to draw bounding boxes around fire and smoke regions, labelling each detection with class name and confidence score. Annotated image is saved to media/results/ and displayed on the same page.
- **On-Demand Model Training**: Retrains all seven ML classifiers through the web interface with live metrics, bar charts, and radar charts.
- **Interactive Dataset Browser**: Displays all 62,630 IoT sensor dataset rows in a searchable, filterable, and paginated interface.
- **User and Admin Management**: Full registration, login, session management, and admin-controlled account activation workflow.

8.5 Hardware and Software Environment

Table 8.3: Development and Testing Environment

Component	Specification
CPU	Intel Core i7-12700H
GPU	NVIDIA GeForce RTX 3050 Ti Laptop GPU
RAM	16 GB
Operating System	Windows 11 64-bit
Python Version	3.10.13 (conda environment: smoke)
CUDA Toolkit	11.2
Django Version	5.2
TensorFlow Version	2.10.0
Scikit-Learn Version	1.6.1
Ultralytics (YOLOv8)	8.4.21
OpenCV	4.8.1.78 (opencv-python-headless)
NumPy	1.26.4 (TF+YOLO compatible)

8.6 System Testing

The purpose of testing is to discover errors and validate that the system meets all specified requirements. Testing was carried out across three levels to ensure correct behaviour of all components and their integration.

8.6.1 Unit Testing

Unit testing validates the internal logic and input-output behaviour of individual components. Each Django view function was tested independently: the registration view was tested with valid and duplicate email inputs; the training view was tested to confirm all seven classifiers train and save correctly; the prediction view was tested with valid 13-feature sensor arrays and out-of-range values; and the CNN prediction view was tested with fire and non-fire image uploads. All decision branches and validation checks were confirmed to produce the expected outputs.

8.6.2 Integration Testing

Integration testing confirms that components work correctly when combined. The full user workflow was tested end-to-end: user registration followed by admin activation, login, dataset browsing, ML model training, sensor-based prediction, and CNN+YOLO image prediction. The integration between the Django ORM and SQLite database, between the trained model .pkl files and the prediction views, and between the CNN classifier and the YOLOv8 detector on the same prediction page were all validated to function without errors.

8.6.3 Functional Testing

Functional testing validates that all features perform as specified by the requirements. Valid input classes were confirmed to be accepted and invalid inputs rejected with appropriate error messages. All application outputs — prediction badges, confidence scores, annotated YOLO images, metric cards, and training charts — were verified to render correctly in Google Chrome.

8.6.4 Sample Test Cases

Table 8.4: Sample Test Cases

S. No	Test Case	Expected Result	Result	Remarks
1	User Registration	Account created with status 'waiting' on valid submission	Pass	Fails if email already exists
2	User Login	Valid credentials redirect to user home dashboard	Pass	Unregistered users cannot log in
3	Dataset View	All 62,630 rows displayed in paginated searchable table	Pass	Fails if dataset file is missing
4	Sensor Prediction	Returns SMOKE DETECTED or NO SMOKE DETECTED with confidence score	Pass	Invalid input values raise validation error
5	Admin Login	Admin credentials redirect to admin dashboard	Pass	Invalid login credentials are rejected
6	Admin Activate User	Selected user account status changes to active	Pass	Non-existent user ID returns error

The successful execution of all test cases confirms that the system is functionally correct and performs reliably under expected conditions.

The presence of proper validation and error handling mechanisms ensures the robustness and security of the application in real-world usage.

8.7 Original Contribution

The following original contributions are made by this project to the field of intelligent building safety systems:

- A dual-module detection architecture that integrates seven classical ML classifiers on IoT sensor data with a MobileNetV2 CNN and a YOLOv8 object detector into a single unified Django web application — an integration not previously demonstrated in the building safety literature.
- Simultaneous CNN classification and YOLOv8 bounding-box fire/smoke detection on the same uploaded image within a single web request, presenting both global confidence output and localised spatial detection results on the same prediction page.
- A fully functional role-based web application with administrator-controlled account activation, on-demand model retraining via the browser interface, interactive IoT dataset browsing across 62,630 records, and real-time sensor-based prediction with selectable classifier — all accessible without any local software installation.
- Empirical validation achieving near-perfect AUC-ROC scores (≥0.999) across all seven ML classifiers on the 62,630-row IoT dataset, and 96.98% validation accuracy for the MobileNetV2 CNN, with GPU-accelerated training providing a 2.3× speedup over CPU baseline.
- In addition to the architectural integration, this project contributes by demonstrating the practical feasibility of combining sensor-based machine learning models with vision-based deep learning models within a single, user-accessible platform.
- Unlike traditional systems that operate in isolation, the proposed system bridges multiple data modalities and presents them through a unified interface, thereby improving decision-making efficiency. This integration highlights how complementary technologies can be leveraged together to enhance reliability and provide a more comprehensive safety monitoring solution.

8.8 Conclusion

This project successfully demonstrates the application of both machine learning and deep learning techniques to the real-world problem of building smoke and fire detection. Two independent but complementary detection modules have been built and integrated into a single Django web application: a sensor-based ML module using seven trained classifiers, and an image-based DL module using a MobileNetV2 CNN trained through transfer learning.

The sensor-based ML module achieves near-perfect accuracy on the 62,630-row IoT sensor dataset, with five classifiers achieving perfect AUC-ROC of 1.0000, AdaBoost at 1.0000, and Logistic Regression at 0.9989. The CNN module achieves 96.98 percent validation accuracy. GPU-accelerated training on the NVIDIA RTX 3050 Ti reduced training time by a factor of 2.3. The integrated YOLOv8 detector further enhances the system by localising fire and smoke regions with bounding boxes and class-level confidence scores on the same prediction page.

Overall, the proposed system demonstrates that combining machine learning and deep learning approaches can significantly enhance the accuracy and reliability of fire and smoke detection systems. By integrating sensor-based prediction with image-based analysis, the system provides both early detection capability and visual confirmation, making it more effective than traditional single-method approaches. The high performance achieved across multiple models further validates the suitability of the proposed methodology for real-world safety applications.

Furthermore, the development of a unified web-based platform ensures ease of access, scalability, and practical usability in various environments such as smart buildings, industrial facilities, and public infrastructure. The system not only delivers accurate predictions but also offers user-friendly interaction and real-time monitoring capabilities. This work lays a strong foundation for future enhancements and large-scale deployment, contributing toward safer and more intelligent building management systems.

8.9 Future Scope

While the current system achieves strong results in a controlled software environment, several directions exist for extending its capabilities toward full production deployment and broader real-world applicability.

1. Real-Time IoT Sensor Integration

The current system accepts manual sensor input through a web form. A natural and high-impact extension would be to connect the prediction pipeline directly to a physical IoT sensor network. Using MQTT or REST-based APIs, live sensor readings from smoke, temperature, humidity, CO, and other environmental sensors deployed across a building could be streamed into the ML prediction module in real time. This would enable continuous, automated monitoring rather than on-demand prediction, making the system suitable for 24/7 building surveillance without human intervention.

2. Live Camera Feed Integration

The CNN and YOLOv8 modules currently operate on uploaded still images. Extending the visual detection pipeline to process live RTSP or WebRTC camera streams would allow real-time frame-by-frame fire and smoke detection. Each frame could be passed through the YOLOv8 model, with detected regions highlighted in a live video feed displayed on the web interface. This would transform the system from a diagnostic tool into an active real-time surveillance system, which is particularly valuable in server rooms, warehouses, and public spaces where continuous visual monitoring is required.

3. Automated Alert and Notification System

Currently, detection results are displayed on the web interface only. Integrating an alert system using services such as Twilio for SMS, SendGrid for email, or Firebase Cloud Messaging for push notifications would allow the system to immediately notify security personnel, building managers, or emergency services when smoke or fire is detected. Alerts could include the detection type, confidence score, sensor readings, and a timestamped snapshot of the annotated image, enabling rapid response even when no one is actively monitoring the dashboard.

4. Mobile Application for Remote Monitoring

Developing a companion mobile application for Android and iOS would allow security personnel to monitor building safety status, receive push notifications on detection events, and view live predictions from any location. The existing Django

REST backend could be extended with token-based API endpoints to serve the mobile app, keeping the core ML and detection logic centralised while making the interface available on handheld devices.

5. Multi-Room and Multi-Zone Sensor Support

The current system processes a single set of 13 sensor readings per prediction request. Future work could extend this to support multiple sensor nodes deployed across different rooms or zones within a building. A building map view could be integrated into the web interface, visually indicating the safety status of each zone in real time, enabling security teams to immediately identify the location of a potential fire event rather than only knowing that a detection has occurred somewhere in the building.

6. Automated Model Retraining Pipeline

As more sensor readings and labelled images are collected over time from real deployments, model performance can be improved through periodic retraining on updated data. An automated retraining pipeline could be implemented using scheduled background tasks (such as Celery with Django) to retrain the ML classifiers and CNN whenever a sufficient volume of new data is accumulated, ensuring that model accuracy continues to improve as the system is used in production.

7. Edge Deployment for Low-Latency Detection

For environments where internet connectivity is unreliable or where detection latency must be minimised, the ML classifiers and a quantised version of the MobileNetV2 CNN could be exported and deployed on edge devices such as Raspberry Pi or NVIDIA Jetson Nano. Edge deployment would allow local inference without dependence on a remote server, improving reliability and response time in critical safety scenarios.

8. Explainability and Confidence Reporting

Future versions of the system could incorporate explainability tools such as SHAP (SHapley Additive explanations) for the ML module and Grad-CAM (Gradient-weighted Class Activation Mapping) for the CNN module. These would provide visual and statistical explanations of why the model made a particular prediction, improving trust and interpretability for end users and enabling engineers to identify and correct any model weaknesses identified during deployment.

REFERENCES

[1] J. Gong, H. Liu, and Q. Tang, "Fire Safety and Emergency Management in Urban Areas," *Journal of Safety Research*, vol. 78, pp. 45–56, 2019.

[2] H. Zhao et al., "Urban multipoint fire disaster emergency simulation based on web information," *International Journal of Disaster Risk Reduction*, vol. 101, p. 104223, 2024.

[3] S. Shaharuddin et al., "The role of IoT sensor in smart building context for indoor fire hazard scenario: A systematic review of interdisciplinary articles," *Internet of Things*, vol. 21, p. 100803, 2023.

[4] S. Chitram, S. Kumar, and S. Thenmalar, "Enhancing Fire and Smoke Detection Using Deep Learning Techniques," *Engineering Proceedings*, vol. 62, no. 1, p. 7, 2024.

[5] I. U. Din et al., "Machine learning in the Internet of Things: Designed techniques for smart cities," *Future Generation Computer Systems*, vol. 100, pp. 826–843, 2019.

[6] E. S. Tomé et al., "An online anomaly detection approach for fault detection on fire alarm systems," *Sensors*, vol. 23, no. 10, p. 4902, 2023.

[7] Deep Contractor, "Smoke Detection Dataset," *Kaggle*. [Online]. Available: https://www.kaggle.com/datasets/deepcontractor/smoke-detection-dataset. [Accessed: May 13, 2024].

[8] S.-Y. Kim and A. Muminov, "Forest fire smoke detection based on deep learning approaches and unmanned aerial vehicle images," *Sensors*, vol. 23, no. 12, p. 5702, 2023.

[9] B. Senthilnayaki et al., "Deep Learning-Based Fire and Smoke Detection System," in *Proc. 2nd Int. Conf. on Emerging Trends in Information Technology and Engineering (ICETITE)*, IEEE, 2024.

[10] S. Khan et al., "DeepSmoke: Deep learning model for smoke detection and segmentation in outdoor environments," *Expert Systems with Applications*, vol. 182, p. 115125, 2021.

[11] K. Avazov et al., "Fire detection method in smart city environments using a deep-learning-based approach," *Electronics*, vol. 11, no. 1, p. 73, 2022.

[12] H. Jabnouni et al., "Machine Learning based Classification for Fire and Smoke Images Recognition," in *Proc. 2022 8th Int. Conf. on Control, Decision and Information Technologies (CoDIT)*, vol. 1, IEEE, 2022.

[13] Y. Valikhujaev, A. Abdusalomov, and Y. I. Cho, "Automatic fire and smoke detection method for surveillance systems based on dilated CNNs," *Atmosphere*, vol. 11, no. 11, p. 1241, 2020.

[14] L.-C. Su et al., "Smart performance-based design for building fire safety: Prediction of smoke motion via AI," *Journal of Building Engineering*, vol. 43, p. 102529, 2021.

[15] M. M. Jahefer, "Deep Learning-Based Fire Detection for Enhanced Safety Systems," *Wasit Journal for Pure Sciences*, vol. 2, no. 4, pp. 45–55, 2023.

[16] Z. Ren, F. Tian, X. Gu, and L. Kang, "Fire Smoke Detection Based on Vision Transformer," in *Proc. 2022 Int. Conf. on Machine Learning and Intelligent Systems Engineering (MLISE)*, IEEE, 2022, doi: 10.1109/MLISE57402.2022.00046.

[17] X. Wang, M. Li, M. Gao, Q. Liu, Z. Li, and L. Kou, "Early smoke and flame detection based on transformer," *Journal of Safety Science and Resilience*, vol. 4, no. 3, pp. 294–304, Sep. 2023, doi: 10.1016/j.jnlssr.2023.06.002.

[18] G. Cheng, Y. Zhou, S. Gao, Y. Li, and H. Yu, "Convolution-enhanced vision transformer network for smoke recognition," *Fire Technology*, vol. 59, no. 2, pp. 925–948, 2023, doi: 10.1007/s10694-023-01378-8.

[19] S. Majid, F. Alenezi, S. Masood, M. Ahmad, E. S. Gündüz, and K. Polat, "Attention based CNN model for fire detection and localization in real-world images," *Expert Systems with Applications*, vol. 189, p. 116114, Mar. 2022, doi: 10.1016/j.eswa.2021.116114.

[20] S. N. Saydirasulovich, M. Mukhiddinov, O. Djuraev, A. Abdusalomov, and Y. I. Cho, "An improved wildfire smoke detection based on YOLOv8 and UAV images," *Sensors*, vol. 23, no. 20, p. 8374, Oct. 2023, doi: 10.3390/s23208374.

[21] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, Jun. 2018, pp. 4510–4520, doi: 10.1109/CVPR.2018.00474.

[22] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.0.0, Ultralytics, Jan. 2023. [Online]. Available: https://github.com/ultralytics/ultralytics.

[23] F. M. Talaat and H. ZainEldin, "An improved fire detection approach based on YOLO-v8 for smart cities," *Neural Computing and Applications*, vol. 35, no. 28, pp. 20939–20954, 2023, doi: 10.1007/s00521-023-08809-1.

[24] K. Muhammad, J. Ahmad, and S. W. Baik, "Early fire detection using convolutional neural networks during surveillance for effective disaster management," *Neurocomputing*, vol. 288, pp. 30–42, 2018, doi: 10.1016/j.neucom.2017.04.083.

[25] V. E. Sathishkumar, J. Cho, M. Subramanian, and O. S. Naren, "Forest fire and smoke detection using deep learning-based learning without forgetting," *Fire Ecology*, vol. 19, no. 1, p. 9, 2023, doi: 10.1186/s42408-022-00165-0.

[26] A. Ramadan, M. H. Behiry, and M. Aly, "Machine learning hybrid dynamic best model selection algorithm for real-time fire prediction using IoT-enabled multi-sensor data in buildings," *Journal of Safety Science and Resilience*, vol. 6, no. 2, pp. 114–126, 2025, doi: 10.1016/j.jnlssr.2025.01.003.

[27] S. Sachar and A. Kumar, "Empowering fire and smoke detection in smart monitoring through deep learning fusion," *International Journal of Information Technology*, vol. 16, pp. 1963–1973, 2024, doi: 10.1007/s41870-023-01630-y.

[28] M. Nadeem, N. Dilshad, N. S. Alghamdi, L. M. Dang, H. Song, J. Nam, and H. Moon, "Visual intelligence in smart cities: A lightweight deep learning model for fire detection in an IoT environment," *Smart Cities*, vol. 6, no. 5, pp. 2245–2259, 2023, doi: 10.3390/smartcities6050103.

[29] C. Esposito, A. Minutolo, R. Megna, F. Forastiere, M. Magliulo, and G. De Pietro, "Early wildfire detection using different machine learning algorithms," *Remote Sensing Applications: Society and Environment*, vol. 35, p. 101184, 2024, doi: 10.1016/j.rsase.2024.101184.

[30] S. T. Seydi, V. Saeidi, B. Kalantar, N. Ueda, and A. A. Halin, "Fire-Net: A deep learning framework for active forest fire detection," *Journal of Sensors*, vol. 2022, p. 8044390, 2022, doi: 10.1155/2022/8044390.